



CREATORZ.HU — MASTER FEJLESZTÉSI PROMPT FÁJL

Verzió: 1.0

Készült: 2026. május

Cél: Magyar UGC tartalomgyártó és márka összekötő platform

Fejlesztő: Claude Code (Anthropic)

Üzemeltető: [Te magad]



HOGYAN HASZNÁLD EZT A FÁJLT

Ez a dokumentum egy **lépésről-lépésre, fázisonkénti fejlesztési útmutató**, amit a Claude Code-nak adsz át. A Claude Code minden fázist **önállóan** végrehajt, és **csak akkor áll meg**, amikor a fázis végén egy 🖐️ **ELLENŐRZÉSI PONT** szakaszhoz ér.

A te dolgod:

1. A 0. fázist (pre-flight setup) **kézzel** elvégzed, mielőtt elindítod a Claude Code-ot
2. Onnantól a Claude Code automatikusan dolgozik
3. Minden 🖐️ ellenőrzési pontnál átnézed az eredményt, és vagy:
 - ✅ Folytatás engedélyezése → "**Mehet a következő fázis**"
 - ❌ Hiba találat → "**Hiba: [leírás]**", és Claude Code javítja, majd újra ellenőrizteti veled

A Claude Code dolga:

- Pontosan végrehajtani minden fázis instrukcióit
 - Sose változtatni a tech stack-en kérdés nélkül
 - Sose hagyni ki lépést
 - Ellenőrzési pontoknál megállni és várni a felhasználói visszajelzést
-



PROJEKT ÖSSZEFOGLALÓ

Domain: creatorz.hu

Brand színek: Fekete (#0A0A0A) + Neon Lime zöld (#A3E635)

Tech stack: Next.js 15 + TypeScript + Tailwind CSS + shadcn/ui + Supabase + Stripe + Resend + Vercel

Hosting: Vercel (free tier indulásnál)

Domain regisztráció: Rackhost.hu

Email: info@creatorz.hu (Rackhost-on)

Stripe fiók típusa: Magyar (Hungary)

Fizetés: HUF-ban, Stripe Subscriptions-on keresztül

Képgenerálás: Replicate FLUX.1 API

Üzleti modell: Directory + subscription

- Brand regisztráció és böngészés: **mindig ingyenes**
- Creator regisztráció: ingyenes vagy 2 990 Ft/hó (admin toggle)
- 7 napos kiemelés: 4 990 Ft

- 30 napos kiemelés: 12 990 Ft
- Brand hirdetésfeladás: ingyenes
- Reviews: Modell A — csak elfogadott pályázat után



FÁZISOK ÁTTEKINTÉSE

Fázis	Cél	Időbecslés	Aktor
0	Pre-flight setup (Windows)	30-60 perc	TE (kézi)
1	Projekt alap és tech stack	4-6 óra	Claude Code
2	Adatbázis schema + auth	4-6 óra	Claude Code
3	Creator profil rendszer	6-8 óra	Claude Code
4	Brand oldal + böngészés	6-8 óra	Claude Code
5	Hirdetésrendszer + pályázás	6-8 óra	Claude Code
6	Stripe Subscriptions + kiemelés	4-6 óra	Claude Code
7	Review rendszer (Modell A)	3-4 óra	Claude Code
8	Admin panel	4-6 óra	Claude Code
9	Képgenerálás + landing page	3-4 óra	Claude Code
10	Social scrape + Cron job	2-3 óra	Claude Code
11	Polish, GDPR, launch	4-6 óra	Claude Code
12	Deployment Vercel-re + DNS	1-2 óra	Claude Code + TE

Teljes fejlesztési idő: kb. 50-70 óra aktív Claude Code-os munka (5-7 hét, napi 6-8 óra ráfordítással)

0. FÁZIS — PRE-FLIGHT SETUP (TE CSINÁLOD!)

 **FONTOS:** ezt a fázist **KÉZZEL** kell elvégezned, **MIELŐTT** elindítod a Claude Code-ot. Minden későbbi fázis erre épül.

LÉNYEGES: "FEJLESZD ELŐBB, DOMAINT VEGYÉL KÉSŐBB"

Az egész platformot felépítheted és kipróbálhatod a SAJÁT GÉPEDEN, 0 forintból, mielőtt bármilyen domaint vagy tárhelyet vennél.

A fejlesztés és tesztelés teljes egészében a gépeden zajlik a `http://localhost:3000` címen. Itt minden funkció teljes értékűen működik:

- Regisztráció, bejelentkezés
- Creator és brand profilok
- Hirdetésfeladás, pályázás
- Stripe fizetés (Test mode — fake kártyával: `4242 4242 4242 4242`)
- Email küldés (Resend free tier)
- Képgenerálás (Replicate)
- Admin panel

Mi NEM kell a lokális fejlesztéshez és teszteléshez:

-  Domain (creatorz.hu) — NEM kell most
-  Tárhely (Rackhost vagy Vercel) — NEM kell most
-  Pénz — minden free tier ingyenes

Mi KELL (mind ingyenes):

-  Node.js (lásd 0.2)
-  Supabase projekt — ez NEM "tárhely", csak a felhős adatbázis (free tier)
-  API kulcsok (Stripe Test mode, Resend, Replicate — mind ingyenes)

A teljes workflow:

Szakasz	Hol fut	Költség
Fejlesztés + tesztelés (1-11. fázis)	A te gépeden (<code>localhost:3000</code>)	0 Ft
Adatbázis	Supabase felhő (free tier)	0 Ft
Tesztfizetések	Stripe Test mode	0 Ft
Élesítés (12. fázis, amikor készen állsz)	Vercel + domain	~1 900 Ft/év

Tehát: végezd el a 0.2-0.8 lépéseket (Node.js, API kulcsok, MCP setup), majd engedd a Claude Code-ot végigmenni az 1-11. fázison. Mindent tesztelj localhost-on. **Csak amikor elégedett vagy, akkor vedd meg a domaint** (lásd 0.1 alább, halasztva) és élesíts a 12. fázisban.

0.1 Domain regisztráció Rackhost-on — « EZT HALASZD A 12. FÁZISRA!

« **FIGYELEM:** Ezt a lépést **NYUGODTAN HAGYD KI MOST**, ha előbb lokálisan akarsz fejleszteni és tesztelni (ajánlott!). A domain csak a 12. fázisban (élesítés) szükséges. Térj vissza ehhez a lépéshez, amikor a localhost-on minden működik és tetszik.

Amikor élesíteni akarsz, akkor jönnek ezek a lépések:

- Menj a <https://www.rackhost.hu> oldalra
- A felső kereső mezőbe írd be: `creatorz`
- Pipáld be a `.hu` végződést
- Kattints a "Keresés" gombra
- Ha a `creatorz.hu` szabad, kattints a "Megrendelés" gombra
- **Csak a domain regisztrációt rendeld meg**, ne pipáld be tárhelyet, WordPress-t, SSL-t
- **Pipáld be** az email szolgáltatást — `info@creatorz.hu` címhez (kb. 1 500 Ft/év)
- Időtartam: ajánlott 2 év (akciósan olcsóbb)
- Add meg a tulajdonosi adatokat (magánszemély vagy cég, ahogy nálad releváns)
- Fizess bankkártyával vagy átutalással
- **Várj 1-3 munkanapot** a `.hu` domain bejegyzésére

Várható összeg: ~3 400 Ft (domain + email, év 1)

Ellenőrzés (csak élesítéskor):

- [] Domain regisztráció megerősítő email megérkezett
- [] Belépés a Rackhost ügyfélfelületre működik (<https://my.rackhost.hu>)
- [] DNS-szerkesztő hozzáférhető a domain alatt

0.2 Windows fejlesztői környezet ellenőrzése

Node.js verzió ellenőrzése

Nyiss egy **PowerShell**-t (Windows + R → `powershell` → Enter), majd futtasd:

```
node --version
```

Elvárás: v20.0.0 vagy újabb (ideálisan v22.x)

Ha nincs telepítve vagy régi:

1. Menj a <https://nodejs.org>-ra
2. Töltsd le az **LTS** verziót (jelenleg v22.x)
3. Telepítsd
4. Indítsd újra a PowerShell-t
5. Ellenőrizd: `node --version`

Git ellenőrzése

```
git --version
```

Elvárás: v2.40+ verzió.

Ha nincs:

1. <https://git-scm.com/download/win>
2. Telepítsd alapbeállításokkal
3. Indítsd újra a PowerShell-t

npm ellenőrzése

```
npm --version
```

Elvárás: v10+ verzió.

0.3 Claude Code telepítése

A **Claude Code** az Anthropic CLI eszköze, ami terminálban fut.

Telepítés

PowerShell-ben:

```
npm install -g @anthropic-ai/claude-code
```

Bejelentkezés

```
claude
```

Az első indításnál:

- Megkér, hogy jelentkez be az Anthropic fiókkal (Pro vagy Max előfizetés szükséges)
- Vagy adj meg egy API kulcsot (akkor pay-as-you-go)

Ajánlott: Anthropic **Max 5x előfizetés** (\$100/hó) — ez biztosítja a folyamatos munkát napi 4-6 órán át, korlátozások nélkül.

Tesztelés

```
claude --version
```

Elvárás: valami verziószám, pl. **2.1.34+** (NE használd a v2.1.100+ verziókat a token-fogyasztási bug miatt!)

Ha v2.1.100+ települt és problémád lenne később, downgradeld:

```
npm install -g @anthropic-ai/claude-code@2.1.34
```

0.4 GitHub repository létrehozása

- Menj a <https://github.com/new> oldalra
- Repository name: **creatorz**
- Description: **Magyar UGC tartalomgyártó platform**

- Privacy: **Private** (kezdetnél)
- **NE inicializáld** README-vel, .gitignore-ral vagy licenccel
- Kattints: "Create repository"

A létrejött URL-t jegyezd fel — szükséged lesz rá, valami ilyesmi:

```
https://github.com/[felhasználónév]/creatorz.git
```

0.5 Külső szolgáltatások beállítása

Most jönnek az API kulcsok. Ezeket egy biztonságos helyen tárold (pl. egy `.txt` fájlban a gépeden, soha NE töltsd fel sehova).

0.5.1 Supabase projekt létrehozása

- Menj a <https://supabase.com> oldalra
- Sign up GitHub-bal
- "New project" gomb
- **Name:** `creatorz`
- **Database password:** generálj egy erős jelszót (mentsd el biztos helyre!)
- **Region:** `Central EU (Frankfurt)` — legközelebbi
- **Pricing plan:** `Free` (most ennyi elég)
- Kattints: "Create new project"
- Várj ~2 percet, amíg a projekt elkészül

Mentsd el ezeket (a `Project Settings` → `API` menüben):

```
SUPABASE_URL = https://[project-ref].supabase.co
SUPABASE_ANON_KEY = eyJhbG... (publikus, hosszú string)
SUPABASE_SERVICE_ROLE_KEY = eyJhbG... (TITKOS, NE oszd meg!)
DATABASE_URL = postgresql://postgres.[project-ref]:[password]@... (a Connection Pooling menüben)
```

0.5.2 Stripe API kulcsok (magyar fiók)

- Lépj be a <https://dashboard.stripe.com>-ra
- Bal felső sarokban válaszd a "Hungary" fiókot (ha többed van)
- **Test mode** legyen ON (jobb felső sarokban Test/Live kapcsoló)
- Menj: `Developers` → `API keys`
- Másold ki:

```
STRIPE_PUBLISHABLE_KEY = pk_test_...
STRIPE_SECRET_KEY = sk_test_...
```

Plusz: Stripe webhook secret — ezt majd a 6. fázis közben fogjuk létrehozni, most még ne foglalkozz vele.

0.5.3 Resend account (email küldéshez)

- Menj a <https://resend.com> oldalra
- Sign up GitHub-bal

- Dashboard → **API Keys** → **Create API Key**
- Name: **creatorz**
- Permission: **Sending access**
- Másold ki:

```
RESEND_API_KEY = re_...
```

Domain hozzáadása (később, a 11. fázisban):

- Dashboard → **Domains** → **Add Domain** → **creatorz.hu**
- DNS rekordok beállítása Rackhost-on (lesz erre is útmutató)

0.5.4 Replicate API kulcs (kép-generáláshoz)

- Menj a <https://replicate.com> oldalra
- Sign up GitHub-bal
- Add meg a bankkártyádat (Settings → Billing) — Replicate **pay-as-you-go**, fogyasztás alapján
- **Töltsd fel** \$10-et (kb. 3 700 Ft) — ez bőven elég ~300+ kép generálásra FLUX.1-gyel
- Settings → **API tokens** → **Create token**
- Name: **creatorz**
- Másold ki:

```
REPLICATE_API_TOKEN = r8_...
```

0.5.5 Anthropic API kulcs (a kép-generáláshoz használt MCP-hez, opcionális)

Ha akarod, hogy a Claude Code közben tudjon "látni" képeket (pl. ellenőrizni az általa generált képet), kell egy Anthropic API kulcs is:

- <https://console.anthropic.com> → API Keys
- Create Key → Name: **creatorz-vision**
- Másold ki:

```
ANTHROPIC_API_KEY = sk-ant-...
```

FIGYELEM: Ha az **ANTHROPIC_API_KEY** környezeti változó be van állítva a gépeden, a Claude Code az **API-t** fogja használni az előfizetésed helyett! Csak akkor állítsd be, ha tudatosan ezt akarod.

0.6 Image generation MCP server beállítása

Ez a kulcs-lépés, hogy a Claude Code **automatikusan** tudjon képeket generálni Replicate-en keresztül.

MCP konfigurációs fájl létrehozása

PowerShell-ben:

```
mkdir -Force "$env:USERPROFILE\.config\claude-code"
notepad "$env:USERPROFILE\.config\claude-code\mcp_servers.json"
```

Másold be ezt a tartalmat (a `REPLICATE_API_TOKEN` helyére írd a saját kulcsodat):

```
{
  "mcpServers": {
    "replicate-flux": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-replicate"
      ],
      "env": {
        "REPLICATE_API_TOKEN": "r8_HELYETTESITSD_A_SAJATODDAL"
      }
    },
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "C:\\Users\\[FELHASZNÁLÓNÉV]\\creatorz"
      ]
    }
  }
}
```

FIGYELEM:

- Cseréld le a `[FELHASZNÁLÓNÉV]`-et a Windows felhasználói nevedre
- Cseréld le a `REPLICATE_API_TOKEN` értéket a saját Replicate kulcsodra

Mentsd el a fájlt.

MCP működésének tesztelése

Indíts egy üres mappában egy új Claude Code session-t:

```
cd "$env:USERPROFILE"
mkdir test-mcp
cd test-mcp
claude
```

A Claude promptba írd:

```
Listázd ki, milyen MCP eszközök vannak elérhetőek.
```

Elvárás: a Claude válaszában szerepelnie kell a `replicate-flux` és `filesystem` szervereknek.

Ha valami nem jó, ellenőrizd:

- Az `mcp_servers.json` szintaxisa helyes-e (valid JSON)
- A Replicate API token beragadt-e a helyére
- A felhasználói név pontosan stimmel-e

0.7 Vercel fiók előkészítése

- Menj a <https://vercel.com> oldalra
- Sign up GitHub-bal (fontos, hogy GitHub-os legyen, mert így tud autodeploy-olni)
- Hagyd ki a "Add a Team" lépést — `Personal` legyen

- Most még semmi mást ne csinálj — a deployment a 12. fázisban lesz

0.8 Munkamappa előkészítése

PowerShell-ben:

```
cd "$env:USERPROFILE"
mkdir creatorz
cd creatorz
```

Itt fogunk dolgozni mostantól. Minden Claude Code parancsot innen indíts.

👉 0. FÁZIS ELLENŐRZÉSI PONT

Mielőtt továbblépsz, ellenőrizd ezeket:

- [] `node --version` legalább v20.x mutatja
- [] `git --version` legalább v2.40+ mutatja
- [] `claude --version` valami verziószámot mutat
- [] GitHub repo `creatorz` létrejött (private)
- [] Supabase projekt készen áll, API kulcsok mentve
- [] Stripe Test mode kulcsok mentve
- [] Resend API kulcs mentve
- [] Replicate API token mentve + \$10 feltöltve
- [] MCP `replicate-flux` szerver tesztelve és működik
- [] `C:\Users\[Te]\creatorz` mappa létrehozva

» Ezek CSAK a 12. fázishoz (élesítés) kellenek — most NYUGODTAN kihagyhatod:

- [] ~~Vercel fiók aktív, GitHub-bal összekötve~~ (halasztva — 12. fázis)
- [] ~~Rackhost-on `creatorz.hu` regisztráció~~ (halasztva — 12. fázis)

Ha a fenti (nem halasztott) pontok ✓, akkor készen állsz a Claude Code indítására és a teljes lokális fejlesztésre.

Indítsd el a Claude Code-ot:

```
cd "$env:USERPROFILE\creatorz"
claude
```

És másold be neki ezt a prompt fájlt egészében, vagy add át fájlként:

```
Olvasd be a creatorz-master-prompt.md fájlt, és kezdj el dolgozni az 1. fázistól.
Minden fázis után állj meg az 👉 ELLENŐRZÉSI PONT-nál és várj a visszajelzésekre.
```



1. FÁZIS — PROJEKT ALAP ÉS TECH STACK

 **Aktor:** Claude Code

 **Időbecslés:** 4-6 óra

 **Cél:** Next.js 15 + TypeScript + Tailwind + shadcn/ui + Supabase + Drizzle + alap layout — minden alpinfrastruktúra felépítve

1.1 Next.js 15 projekt inicializálása

A `C:\Users\[Te]\creatorz` mappában futtasd:

```
npx create-next-app@latest . --typescript --tailwind --app --no-eslint --no-src-dir --import-alias "@/*" --turbo
```

Válaszok az interaktív kérdésekre:

- TypeScript: **Yes**
- Tailwind CSS: **Yes**
- App Router: **Yes**
- src/ directory: **No**
- Customize import alias: **Yes**, value: `@/*`
- Turbopack: **Yes**

A telepítés után:

```
npm run dev
```

Nyisd meg a böngészőben: `http://localhost:3000`

Várt eredmény: Next.js default oldal megjelenik.

Állítsd le a fejlesztő szerveret: `Ctrl + C`

1.2 Tech stack csomagok telepítése

```
npm install drizzle-orm @supabase/supabase-js @supabase/ssr stripe resend zod react-hook-form @hookform/resolvers lucide-react date-fns @vercel/cron  
npm install -D drizzle-kit @types/node tsx eslint eslint-config-next prettier prettier-plugin-tailwindcss
```

1.3 shadcn/ui telepítése

```
npx shadcn@latest init
```

Válaszok:

- Style: **New York**

- Base color: **Neutral**

- CSS variables: **Yes**

Komponensek installálása (kezdő szett):

```
npx shadcn@latest add button card input label textarea select badge avatar dialog dropdown-menu  
form sheet skeleton sonner tabs tooltip alert separator switch
```

1.4 Projekt struktúra létrehozása

Hozd létre az alábbi mappákat és üres `.gitkeep` fájlokat:

```
creatorz/
├── app/
│   ├── (auth)/
│   │   ├── login/
│   │   ├── register/
│   │   └── layout.tsx
│   ├── (dashboard)/
│   │   ├── creator/
│   │   ├── brand/
│   │   ├── admin/
│   │   └── layout.tsx
│   ├── (public)/
│   │   ├── creators/
│   │   ├── ads/
│   │   ├── about/
│   │   └── layout.tsx
│   ├── api/
│   │   ├── auth/
│   │   ├── stripe/
│   │   ├── cron/
│   │   ├── upload/
│   │   └── webhooks/
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── ui/           ← shadcn/ui ide telepít
│   ├── layout/
│   ├── creator/
│   ├── brand/
│   ├── shared/
│   └── admin/
├── lib/
│   ├── db/
│   │   ├── schema.ts
│   │   ├── index.ts
│   │   └── migrations/
│   ├── supabase/
│   │   ├── client.ts
│   │   ├── server.ts
│   │   └── middleware.ts
│   ├── stripe/
│   │   ├── client.ts
│   │   └── helpers.ts
│   ├── resend/
│   │   ├── client.ts
│   │   └── templates/
│   ├── replicate/
│   │   └── client.ts
│   ├── utils.ts
│   └── constants.ts
├── public/
│   ├── images/
│   └── icons/
├── types/
│   └── database.ts
├── middleware.ts
├── drizzle.config.ts
├── .env.local
├── .env.example
├── README.md
└── CLAUDE.md
```

1.5 Környezeti változók beállítása

Hozz létre egy `.env.local` fájlt a következő tartalommal — **a kulcsokat én megadom kérdezve a felhasználótól**, de a struktúra ez:

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=
DATABASE_URL=

# Stripe (Test mode kulcsok kezdéskor)
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=

# Resend
RESEND_API_KEY=
EMAIL_FROM=Creatorz <hello@creatorz.hu>

# Replicate (kép-generáláshoz)
REPLICATE_API_TOKEN=

# App
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_APP_NAME=Creatorz

# Admin (induló admin email – később adatbázisban kezelve)
ADMIN_EMAIL=

# Cron secret (Vercel Cron biztosításához – generálj erős random stringet)
CRON_SECRET=

# Feature flags (admin által módosítható, de default érték itt)
NEXT_PUBLIC_CREATOR_SUBSCRIPTION_ENABLED=false
NEXT_PUBLIC_CREATOR_SUBSCRIPTION_PRICE_HUF=2990
NEXT_PUBLIC_FEATURE_7DAY_PRICE_HUF=4990
NEXT_PUBLIC_FEATURE_30DAY_PRICE_HUF=12990
```

Claude Code: kérdezd meg a felhasználót egyenként az értékekért, és töltsd be őket. NE futtasd a projektet, amíg a `.env.local` nincs kitöltve.

Hozz létre egy `.env.example` fájlt is ugyanezzel a struktúrával, de üres értékekkel (ezt commit-oljuk).

Frissítsd a `.gitignore` fájlt, hogy tartalmazza:

```
.env.local
.env*.local
node_modules
.next
.vercel
.DS_Store
```

1.6 Brand színek beállítása Tailwind-ben

Frissítsd a `app/globals.css` fájlt:

```
@tailwind base;
@tailwind components;
@tailwind utilities;

@layer base {
  :root {
    /* Creatorz Brand Colors */
    --background: 0 0% 100%;
    --foreground: 0 0% 4%;          /* #0A0A0A főszín */

    --card: 0 0% 100%;
    --card-foreground: 0 0% 4%;

    --popover: 0 0% 100%;
    --popover-foreground: 0 0% 4%;

    --primary: 0 0% 4%;            /* fekete */
    --primary-foreground: 0 0% 100%;

    --secondary: 0 0% 96%;
    --secondary-foreground: 0 0% 4%;

    --muted: 0 0% 96%;
    --muted-foreground: 0 0% 45%;

    --accent: 84 81% 56%;          /* #A3E635 neon lime */
    --accent-foreground: 0 0% 4%;

    --destructive: 0 84% 60%;
    --destructive-foreground: 0 0% 100%;

    --border: 0 0% 90%;
    --input: 0 0% 90%;
    --ring: 84 81% 56%;           /* lime focus ring */

    --radius: 0.75rem;
  }

  .dark {
    --background: 0 0% 4%;          /* #0A0A0A */
    --foreground: 0 0% 100%;

    --card: 0 0% 8%;
    --card-foreground: 0 0% 100%;

    --popover: 0 0% 8%;
    --popover-foreground: 0 0% 100%;

    --primary: 84 81% 56%;          /* lime az elsődleges sötét módban */
    --primary-foreground: 0 0% 4%;

    --secondary: 0 0% 14%;
    --secondary-foreground: 0 0% 100%;

    --muted: 0 0% 14%;
    --muted-foreground: 0 0% 64%;

    --accent: 84 81% 56%;
    --accent-foreground: 0 0% 4%;

    --destructive: 0 62% 50%;
    --destructive-foreground: 0 0% 100%;

    --border: 0 0% 18%;
    --input: 0 0% 18%;
  }
}
```

```
--ring: 84 81% 56%;
}
}

@layer base {
  * {
    @apply border-border;
  }
  body {
    @apply bg-background text-foreground antialiased;
    font-feature-settings: "rlig" 1, "calt" 1;
  }
}

/* Custom utilities */
@layer utilities {
  .text-balance {
    text-wrap: balance;
  }

  /* Lime glow effect for featured items */
  .lime-glow {
    box-shadow: 0 0 20px rgba(163, 230, 53, 0.4);
  }

  /* Subtle dark gradient backgrounds */
  .dark-gradient {
    background: linear-gradient(135deg, #0A0A0A 0%, #1a1a1a 100%);
  }
}
```

1.7 Font beállítása

Frissítsd a `app/layout.tsx` fájlt, hogy az **Inter** font-ot használja:

```
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";
import { Toaster } from "@components/ui/sonner";

const inter = Inter({
  subsets: ["latin", "latin-ext"],
  variable: "--font-inter",
});

export const metadata: Metadata = {
  title: {
    default: "Creatorz – Magyar UGC tartalomgyártó platform",
    template: "%s | Creatorz",
  },
  description: "Találd meg a tökéletes magyar UGC tartalomgyártót a márkádhoz, vagy regisztrálj creatorként és kezdj el dolgozni magyar brandekkel.",
  keywords: ["UGC", "tartalomgyártó", "magyar creator", "influencer marketing", "márka tartalom"],
  authors: [{ name: "Creatorz" }],
  metadataBase: new URL(process.env.NEXT_PUBLIC_APP_URL || "http://localhost:3000"),
  openGraph: {
    type: "website",
    locale: "hu_HU",
    url: process.env.NEXT_PUBLIC_APP_URL,
    siteName: "Creatorz",
    images: [
      {
        url: "/og-image.png",
        width: 1200,
        height: 630,
        alt: "Creatorz",
      },
    ],
  },
  robots: {
    index: true,
    follow: true,
  },
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="hu" className={inter.variable}>
      <body className="min-h-screen font-sans">
        {children}
        <Toaster richColors position="top-right" />
      </body>
    </html>
  );
}
```

1.8 Alapvető constants

`lib/constants.ts`:


```
export const APP_NAME = "Creatorz";
export const APP_DOMAIN = "creatorz.hu";
export const APP_URL = process.env.NEXT_PUBLIC_APP_URL || "https://creatorz.hu";

// Categories (creator stílusok)
export const CREATOR_CATEGORIES = [
  { value: "gasztro", label: "Gasztro", emoji: "[food]" },
  { value: "utazas", label: "Utazás", emoji: "[travel]" },
  { value: "divat", label: "Divat", emoji: "[fashion]" },
  { value: "sport", label: "Sport", emoji: "[sport]" },
  { value: "beauty", label: "Beauty", emoji: "[beauty]" },
  { value: "lifestyle", label: "Lifestyle", emoji: "[star]" },
  { value: "tech", label: "Tech", emoji: "[tech]" },
  { value: "otthon", label: "Otthon", emoji: "[home]" },
  { value: "anyukak", label: "Anyukáknak", emoji: "[baby]" },
  { value: "auto", label: "Autó", emoji: "[car]" },
  { value: "fitness", label: "Fitness", emoji: "[fit]" },
  { value: "wellness", label: "Wellness", emoji: "[yoga]" },
  { value: "vendeglatas", label: "Vendéglátás", emoji: "[dining]" },
  { value: "gyerek", label: "Gyerek", emoji: "[toy]" },
  { value: "allatok", label: "Állatok", emoji: "[paws]" },
  { value: "egyeb", label: "Egyéb", emoji: "[box]" },
] as const;

export const HUNGARIAN_COUNTIES = [
  "Bács-Kiskun", "Baranya", "Békés", "Borsod-Abaúj-Zemplén", "Budapest",
  "Csongrád-Csanád", "Fejér", "Győr-Moson-Sopron", "Hajdú-Bihar", "Heves",
  "Jász-Nagykun-Szolnok", "Komárom-Esztergom", "Nógrád", "Pest", "Somogy",
  "Szabolcs-Szatmár-Bereg", "Tolna", "Vas", "Veszprém", "Zala",
] as const;

export const CONTENT_TYPES = [
  { value: "video", label: "Videó" },
  { value: "photo", label: "Fotó" },
  { value: "both", label: "Videó és fotó" },
] as const;

export const USAGE_RIGHTS = [
  { value: "organic", label: "Organic social media" },
  { value: "paid_ads", label: "Paid ads (fizetett hirdetés)" },
  { value: "perpetual", label: "Perpetual (örökös jog)" },
] as const;

export const LANGUAGES = [
  { value: "hu", label: "Magyar" },
  { value: "en", label: "Angol" },
  { value: "de", label: "Német" },
  { value: "ro", label: "Román" },
  { value: "sk", label: "Szlovák" },
] as const;
```

1.9 CLAUDE.md fájl létrehozása

Hozz létre egy **CLAUDE.md** fájlt a repo gyökerében — ez a Claude Code-nak szól, és minden session elején beolvassa:

```
# Creatorz.hu – Project Context

## Mit építünk
Magyar UGC tartalomgyártó platform – directory + subscription modell.
Cégeknek ingyenes a böngészés és kapcsolatfelvétel.
Creatoroknak ingyenes vagy 2 990 Ft/hó (admin toggle).
Kiemelés vásárlás: 7 nap = 4 990 Ft, 30 nap = 12 990 Ft.
Brand hirdetés feladása ingyenes, creator pályázhat rá.
Review-k Modell A szerint: csak elfogadott pályázat után írható.

## Tech stack
- Next.js 15 (App Router) + TypeScript
- Tailwind CSS + shadcn/ui (New York style, Neutral base)
- Supabase (PostgreSQL + Auth + Storage)
- Drizzle ORM
- Stripe Subscriptions (HUF, magyar fiók)
- Resend (emailek)
- Replicate FLUX.1 (kép-generálás)
- Vercel hosting

## Színek
- Háttér: #0A0A0A (fekete)
- Accent: #A3E635 (neon lime)
- Háttér világos: fehér + neutral szürkék

## Konvenciók
- Fájlnevek: kebab-case (creator-profile.tsx)
- Komponensnevek: PascalCase (CreatorProfile)
- Server Components default, "use client" csak ahol kell
- Drizzle schema mindig `lib/db/schema.ts`-ben
- Server Actions a `app/actions/` mappában
- Env változók TypeScript-tel ellenőrizve: `lib/env.ts`
- Mindig magyar nyelvű UI szövegek
- A felhasználói facing dátumok magyar formátumban (2026.05.27.)
- Pénzek mindig HUF-ban (Ft), thousand separator szókód: "2 990 Ft"

## Adatbázis főtáblák
users, creator_profiles, brand_profiles, portfolio_items,
ads, ad_applications, collaborations, reviews, review_responses,
subscriptions, feature_purchases, settings (admin), notifications

## Fontos szabályok
- SOHA ne commit-olj API kulcsot
- Mindig type-safe Server Actions (zod validáció)
- Sose használj browser storage-t (localStorage/sessionStorage) – Supabase session
- Magyar nyelv: minden UI szöveg, hibaüzenet, email
- Mobile-first: kezdés mobil layouttal
```

1.10 Drizzle config

`drizzle.config.ts`:

```
import { defineConfig } from "drizzle-kit";

export default defineConfig({
  schema: "./lib/db/schema.ts",
  out: "./lib/db/migrations",
  dialect: "postgresql",
  dbCredentials: {
    url: process.env.DATABASE_URL!,
  },
  verbose: true,
  strict: true,
});
```

1.11 Git commit

```
git init
git add .
git commit -m "Initial setup: Next.js 15 + Tailwind + shadcn/ui + Supabase + Drizzle"
git branch -M main
git remote add origin https://github.com/[FELHASZNÁLÓNÉV]/creatorz.git
git push -u origin main
```



1. FÁZIS ELLENŐRZÉSI PONT

Ellenőrizd a következőket:

- [] `npm run dev` hibamentesen indul, `http://localhost:3000` betöltődik
- [] A default Next.js oldal megjelenik
- [] Nincs TypeScript hiba: `npx tsc --noEmit`
- [] `.env.local` ki van töltve és működik
- [] `CLAUDE.md` létezik és tartalmaz
- [] GitHub repo `main` branch-en a kód
- [] `components/ui/` mappában a shadcn komponensek

Ha mind ✓: írd vissza a Claude Code-nak: **"Mehet a 2. fázis"**

Ha hiba van: **"Hiba: [leírás]"**

2. FÁZIS — ADATBÁZIS SCHEMA + AUTH

 **Aktor:** Claude Code

 **Időbecslés:** 4-6 óra

 **Cél:** Drizzle schema teljes, Supabase Auth integrálva, role-választás működik

2.1 Drizzle schema létrehozása

`lib/db/schema.ts` — **TELJES schema**, minden táblával:

```

import {
  pgTable, uuid, text, varchar, integer, boolean, timestamp,
  numeric, jsonb, pgEnum, index, uniqueIndex, primaryKey,
} from "drizzle-orm/pg-core";
import { relations } from "drizzle-orm";

// ===== ENUMS =====
export const userRoleEnum = pgEnum("user_role", ["creator", "brand", "admin"]);
export const adStatusEnum = pgEnum("ad_status", ["pending", "active", "closed", "rejected"]);
export const applicationStatusEnum = pgEnum("application_status", ["pending", "accepted", "rejected", "withdrawn"]);
export const subscriptionStatusEnum = pgEnum("subscription_status", ["active", "past_due", "cancelled", "unpaid", "incomplete"]);
export const featureTypeEnum = pgEnum("feature_type", ["7day", "30day"]);
export const contentTypeEnum = pgEnum("content_type", ["video", "photo", "both"]);
export const portfolioTypeEnum = pgEnum("portfolio_type", ["video", "photo"]);

// ===== USERS =====
export const users = pgTable("users", {
  id: uuid("id").primaryKey().defaultRandom(),
  authId: uuid("auth_id").notNull().unique(), // Supabase Auth user ID
  email: text("email").notNull().unique(),
  role: userRoleEnum("role").notNull(),
  approved: boolean("approved").notNull().default(false),
  suspended: boolean("suspended").notNull().default(false),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  updatedAt: timestamp("updated_at").notNull().defaultNow(),
  lastLoginAt: timestamp("last_login_at"),
}, (table) => ({
  emailIdx: uniqueIndex("users_email_idx").on(table.email),
  authIdx: uniqueIndex("users_auth_id_idx").on(table.authId),
}));

// ===== CREATOR PROFILES =====
export const creatorProfiles = pgTable("creator_profiles", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }).unique(),
  username: varchar("username", { length: 50 }).notNull().unique(),
  displayName: varchar("display_name", { length: 100 }).notNull(),
  avatarUrl: text("avatar_url"),
  bannerUrl: text("banner_url"),
  bio: text("bio"), // max 500 chars enforced in zod
  city: varchar("city", { length: 100 }),
  county: varchar("county", { length: 50 }),
  age: integer("age"),
  gender: varchar("gender", { length: 20 }),
  categories: jsonb("categories").$type<string[]>().notNull().default([]),
  languages: jsonb("languages").$type<string[]>().notNull().default(["hu"]),
  equipment: jsonb("equipment").$type<{
    phone?: string;
    camera?: string;
    microphone?: string;
    editing?: string;
  }>(),
});

// Social media
instagramUrl: text("instagram_url"),
instagramFollowers: integer("instagram_followers"),
instagramVerified: boolean("instagram_verified").notNull().default(false),
instagramLastChecked: timestamp("instagram_last_checked"),
tiktokUrl: text("tiktok_url"),
tiktokFollowers: integer("tiktok_followers"),
tiktokVerified: boolean("tiktok_verified").notNull().default(false),
tiktokLastChecked: timestamp("tiktok_last_checked"),
facebookUrl: text("facebook_url"),

```

```

facebookFollowers: integer("facebook_followers"),
facebookVerified: boolean("facebook_verified").notNull().default(false),
facebookLastChecked: timestamp("facebook_last_checked"),
youtubeUrl: text("youtube_url"),
youtubeSubscribers: integer("youtube_subscribers"),
youtubeVerified: boolean("youtube_verified").notNull().default(false),
youtubeLastChecked: timestamp("youtube_last_checked"),

// Rate card (JSON array)
rateCard: jsonb("rate_card").$type<Array<{
  service: string;
  priceHuf: number;
  description?: string;
}>>().notNull().default([]),

// Featured status
isFeatured: boolean("is_featured").notNull().default(false),
featuredUntil: timestamp("featured_until"),
isAdminFeatured: boolean("is_admin_featured").notNull().default(false),

// Stats (cached for performance)
reviewCount: integer("review_count").notNull().default(0),
averageRating: numeric("average_rating", { precision: 3, scale: 2 }),

createdAt: timestamp("created_at").notNull().defaultNow(),
updatedAt: timestamp("updated_at").notNull().defaultNow(),
}, (table) => ({
  usernameIdx: uniqueIndex("creator_profiles_username_idx").on(table.username),
  featuredIdx: index("creator_profiles_featured_idx").on(table.isFeatured),
  categoriesIdx: index("creator_profiles_categories_idx").on(table.categories),
}));

// ===== BRAND PROFILES =====
export const brandProfiles = pgTable("brand_profiles", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }).unique(),
  companyName: varchar("company_name", { length: 200 }).notNull(),
  websiteUrl: text("website_url"),
  logoUrl: text("logo_url"),
  contactName: varchar("contact_name", { length: 100 }),
  contactPhone: varchar("contact_phone", { length: 30 }),
  // Részletes adatok (kötelező csak hirdetésfeladásnál)
  taxNumber: varchar("tax_number", { length: 30 }),
  address: text("address"),
  industry: varchar("industry", { length: 100 }),
  description: text("description"),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  updatedAt: timestamp("updated_at").notNull().defaultNow(),
}, (table) => ({
  companyNameIdx: index("brand_profiles_company_idx").on(table.companyName),
}));

// ===== PORTFOLIO ITEMS =====
export const portfolioItems = pgTable("portfolio_items", {
  id: uuid("id").primaryKey().defaultRandom(),
  creatorId: uuid("creator_id").notNull().references(() => creatorProfiles.id, { onDelete: "casca
de" }),
  type: portfolioTypeEnum("type").notNull(),
  url: text("url").notNull(),
  thumbnailUrl: text("thumbnail_url"),
  title: varchar("title", { length: 200 }),
  description: text("description"),
  categories: jsonb("categories").$type<string[]>().notNull().default([]),
  sortOrder: integer("sort_order").notNull().default(0),
  createdAt: timestamp("created_at").notNull().defaultNow(),
}, (table) => ({

```

```

    creatorIdx: index("portfolio_creator_idx").on(table.creatorId),
  }));

// ===== ADS (BRAND HIRDETÉSEK) =====
export const ads = pgTable("ads", {
  id: uuid("id").primaryKey().defaultRandom(),
  brandId: uuid("brand_id").notNull().references(() => brandProfiles.id, { onDelete: "cascade" })
,
  title: varchar("title", { length: 80 }).notNull(),
  description: text("description").notNull(),
  categories: jsonb("categories").$type<string[]>().notNull().default([]),
  contentType: contentTypeEnum("content_type").notNull(),
  itemCount: integer("item_count").notNull().default(1),
  budgetMinHuf: integer("budget_min_huf").notNull(),
  budgetMaxHuf: integer("budget_max_huf").notNull(),
  deadline: timestamp("deadline").notNull(),
  location: varchar("location", { length: 200 }),
  usageRights: varchar("usage_rights", { length: 50 }).notNull(),
  referenceLinks: jsonb("reference_links").$type<string[]>().notNull().default([]),
  status: adStatusEnum("status").notNull().default("pending"),
  rejectionReason: text("rejection_reason"),
  isFeatured: boolean("is_featured").notNull().default(false),
  applicationCount: integer("application_count").notNull().default(0),
  viewCount: integer("view_count").notNull().default(0),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  approvedAt: timestamp("approved_at"),
  closedAt: timestamp("closed_at"),
}, (table) => ({
  brandIdx: index("ads_brand_idx").on(table.brandId),
  statusIdx: index("ads_status_idx").on(table.status),
  categoriesIdx: index("ads_categories_idx").on(table.categories),
  deadlineIdx: index("ads_deadline_idx").on(table.deadline),
}));

// ===== AD APPLICATIONS (CREATOR PÁLYÁZATOK) =====
export const adApplications = pgTable("ad_applications", {
  id: uuid("id").primaryKey().defaultRandom(),
  adId: uuid("ad_id").notNull().references(() => ads.id, { onDelete: "cascade" }),
  creatorId: uuid("creator_id").notNull().references(() => creatorProfiles.id, { onDelete: "casca
de" }),
  message: text("message").notNull(),
  proposedPriceHuf: integer("proposed_price_huf").notNull(),
  status: applicationStatusEnum("status").notNull().default("pending"),
  rejectionReason: text("rejection_reason"),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  respondedAt: timestamp("responded_at"),
}, (table) => ({
  adIdx: index("applications_ad_idx").on(table.adId),
  creatorIdx: index("applications_creator_idx").on(table.creatorId),
  statusIdx: index("applications_status_idx").on(table.status),
  uniqueIdx: uniqueIndex("applications_unique_idx").on(table.adId, table.creatorId),
}));

// ===== COLLABORATIONS (ELFOGADOTT PÁLYÁZATOKBÓL) =====
export const collaborations = pgTable("collaborations", {
  id: uuid("id").primaryKey().defaultRandom(),
  adId: uuid("ad_id").notNull().references(() => ads.id, { onDelete: "set null" }),
  applicationId: uuid("application_id").references(() => adApplications.id, { onDelete: "set
null" }),
  brandId: uuid("brand_id").notNull().references(() => brandProfiles.id, { onDelete: "cascade" })
,
  creatorId: uuid("creator_id").notNull().references(() => creatorProfiles.id, { onDelete: "casca
de" }),
  acceptedAt: timestamp("accepted_at").notNull().defaultNow(),
  reviewEmailSentAt: timestamp("review_email_sent_at"),
  reviewToken: text("review_token").unique(), // for token-based review submission

```

```

    status: varchar("status", { length: 30 }).notNull().default("active"),
    // status: active / review_pending / reviewed / closed
  }, (table) => ({
    brandIdx: index("collab_brand_idx").on(table.brandId),
    creatorIdx: index("collab_creator_idx").on(table.creatorId),
    tokenIdx: uniqueIndex("collab_token_idx").on(table.reviewToken),
  }));

// ===== REVIEWS =====
export const reviews = pgTable("reviews", {
  id: uuid("id").primaryKey().defaultRandom(),
  collaborationId: uuid("collaboration_id").notNull().references(() => collaborations.id, { onDelete: "cascade" }).unique(),
  brandId: uuid("brand_id").notNull().references(() => brandProfiles.id, { onDelete: "cascade" }),
  creatorId: uuid("creator_id").notNull().references(() => creatorProfiles.id, { onDelete: "cascade" }),
  overallRating: integer("overall_rating").notNull(), // 1-5
  communicationRating: integer("communication_rating").notNull(),
  qualityRating: integer("quality_rating").notNull(),
  deadlineRating: integer("deadline_rating").notNull(),
  text: text("text").notNull(),
  reported: boolean("reported").notNull().default(false),
  hidden: boolean("hidden").notNull().default(false),
  editedUntil: timestamp("edited_until"),
  locked: boolean("locked").notNull().default(false),
  createdAt: timestamp("created_at").notNull().defaultNow(),
}, (table) => ({
  creatorIdx: index("reviews_creator_idx").on(table.creatorId),
  brandIdx: index("reviews_brand_idx").on(table.brandId),
  hiddenIdx: index("reviews_hidden_idx").on(table.hidden),
}));

// ===== REVIEW RESPONSES (CREATOR VÁLASZ) =====
export const reviewResponses = pgTable("review_responses", {
  id: uuid("id").primaryKey().defaultRandom(),
  reviewId: uuid("review_id").notNull().references(() => reviews.id, { onDelete: "cascade" }).unique(),
  creatorId: uuid("creator_id").notNull().references(() => creatorProfiles.id, { onDelete: "cascade" }),
  text: text("text").notNull(),
  editedUntil: timestamp("edited_until"),
  createdAt: timestamp("created_at").notNull().defaultNow(),
});

// ===== MESSAGES (BRAND → CREATOR) =====
export const messages = pgTable("messages", {
  id: uuid("id").primaryKey().defaultRandom(),
  fromUserId: uuid("from_user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  toUserId: uuid("to_user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  subject: varchar("subject", { length: 200 }),
  body: text("body").notNull(),
  budgetHint: integer("budget_hint"),
  read: boolean("read").notNull().default(false),
  createdAt: timestamp("created_at").notNull().defaultNow(),
}, (table) => ({
  toIdx: index("messages_to_idx").on(table.toUserId),
  fromIdx: index("messages_from_idx").on(table.fromUserId),
}));

// ===== SUBSCRIPTIONS =====
export const subscriptions = pgTable("subscriptions", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }).unique(),
  stripeCustomerId: text("stripe_customer_id").notNull().unique(),
  stripeSubscriptionId: text("stripe_subscription_id").unique(),

```



```

status: subscriptionStatusEnum("status").notNull(),
currentPeriodStart: timestamp("current_period_start"),
currentPeriodEnd: timestamp("current_period_end"),
cancelAtPeriodEnd: boolean("cancel_at_period_end").notNull().default(false),
canceledAt: timestamp("canceled_at"),
createdAt: timestamp("created_at").notNull().defaultNow(),
updatedAt: timestamp("updated_at").notNull().defaultNow(),
}, (table) => ({
  stripeCustomerId: uniqueIndex("subs_stripe_customer_idx").on(table.stripeCustomerId),
}));

// ===== FEATURE PURCHASES (kiemelés vásárlás) =====
export const featurePurchases = pgTable("feature_purchases", {
  id: uuid("id").primaryKey().defaultRandom(),
  creatorId: uuid("creator_id").notNull().references(() => creatorProfiles.id, { onDelete: "cascade" }),
  type: featureTypeEnum("type").notNull(),
  stripePaymentIntentId: text("stripe_payment_intent_id").notNull().unique(),
  amountHuf: integer("amount_huf").notNull(),
  startsAt: timestamp("starts_at").notNull(),
  endsAt: timestamp("ends_at").notNull(),
  createdAt: timestamp("created_at").notNull().defaultNow(),
}, (table) => ({
  creatorIdx: index("features_creator_idx").on(table.creatorId),
  activeIdx: index("features_active_idx").on(table.endsAt),
}));

// ===== SETTINGS (ADMIN GLOBAL TOGGLES) =====
export const settings = pgTable("settings", {
  key: varchar("key", { length: 100 }).primaryKey(),
  value: jsonb("value").notNull(),
  description: text("description"),
  updatedAt: timestamp("updated_at").notNull().defaultNow(),
});

// ===== NOTIFICATIONS =====
export const notifications = pgTable("notifications", {
  id: uuid("id").primaryKey().defaultRandom(),
  userId: uuid("user_id").notNull().references(() => users.id, { onDelete: "cascade" }),
  type: varchar("type", { length: 50 }).notNull(),
  title: varchar("title", { length: 200 }).notNull(),
  body: text("body"),
  link: text("link"),
  read: boolean("read").notNull().default(false),
  createdAt: timestamp("created_at").notNull().defaultNow(),
}, (table) => ({
  userIdx: index("notifications_user_idx").on(table.userId),
  readIdx: index("notifications_read_idx").on(table.read),
}));

// ===== RELATIONS =====
export const usersRelations = relations(users, ({ one, many }) => ({
  creatorProfile: one(creatorProfiles),
  brandProfile: one(brandProfiles),
  subscription: one(subscriptions),
  sentMessages: many(messages, { relationName: "from" }),
  receivedMessages: many(messages, { relationName: "to" }),
  notifications: many(notifications),
}));

export const creatorProfilesRelations = relations(creatorProfiles, ({ one, many }) => ({
  user: one(users, { fields: [creatorProfiles.userId], references: [users.id] }),
  portfolio: many(portfolioItems),
  applications: many(adApplications),
  collaborations: many(collaborations),
  reviews: many(reviews),
}));

```

```

    featurePurchases: many(featurePurchases),
  }));

export const brandProfilesRelations = relations(brandProfiles, ({ one, many }) => ({
  user: one(users, { fields: [brandProfiles.userId], references: [users.id] }),
  ads: many(ads),
  collaborations: many(collaborations),
  reviewsGiven: many(reviews),
}));

export const adsRelations = relations(ads, ({ one, many }) => ({
  brand: one(brandProfiles, { fields: [ads.brandId], references: [brandProfiles.id] }),
  applications: many(adApplications),
}));

export const adApplicationsRelations = relations(adApplications, ({ one }) => ({
  ad: one(ads, { fields: [adApplications.adId], references: [ads.id] }),
  creator: one(creatorProfiles, { fields: [adApplications.creatorId], references: [creatorProfiles.id] }),
}));

export const collaborationsRelations = relations(collaborations, ({ one }) => ({
  ad: one(ads, { fields: [collaborations.adId], references: [ads.id] }),
  application: one(adApplications, { fields: [collaborations.applicationId], references: [adApplications.id] }),
  brand: one(brandProfiles, { fields: [collaborations.brandId], references: [brandProfiles.id] }),
  creator: one(creatorProfiles, { fields: [collaborations.creatorId], references: [creatorProfiles.id] }),
  review: one(reviews),
}));

export const reviewsRelations = relations(reviews, ({ one }) => ({
  collaboration: one(collaborations, { fields: [reviews.collaborationId], references: [collaborations.id] }),
  brand: one(brandProfiles, { fields: [reviews.brandId], references: [brandProfiles.id] }),
  creator: one(creatorProfiles, { fields: [reviews.creatorId], references: [creatorProfiles.id] }),
  response: one(reviewResponses),
}));

```

2.2 Drizzle migration generálása

```
npx drizzle-kit generate
```

Ez létrehoz egy SQL migration fájlt a `lib/db/migrations/` mappában.

2.3 Migration futtatása Supabase-ben

Két lehetőség:

Opció A: drizzle-kit push (gyorsabb, fejlesztéshez)

```
npx drizzle-kit push
```

Opció B: SQL másolás (megbízhatóbb)

- Nyisd meg a `lib/db/migrations/0000_*.sql` fájlt

- Másold ki a teljes tartalmát
- Menj a Supabase dashboard → SQL Editor
- Új query → beillesztés → "Run"
- Várj a sikeres futásra

2.4 Drizzle client setup

`lib/db/index.ts`:

```
import { drizzle } from "drizzle-orm/postgres-js";
import postgres from "postgres";
import * as schema from "./schema";

const connectionString = process.env.DATABASE_URL!;

// Disable prefetch as it is not supported for "Transaction" pool mode
const client = postgres(connectionString, { prepare: false });
export const db = drizzle(client, { schema });

export * from "./schema";
```

2.5 Supabase clientek

`lib/supabase/client.ts` (browser):

```
import { createBrowserClient } from "@supabase/ssr";

export function createClient() {
  return createBrowserClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}
```

`lib/supabase/server.ts` (server components):

```
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";

export async function createClient() {
  const cookieStore = await cookies();

  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll();
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            );
          } catch {
            // The `setAll` method was called from a Server Component.
          }
        },
      },
    }
  );
}
```

lib/supabase/middleware.ts :

```
import { createServerClient } from "@supabase/ssr";
import { NextResponse, type NextRequest } from "next/server";

export async function updateSession(request: NextRequest) {
  let supabaseResponse = NextResponse.next({ request });

  const supabase = createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return request.cookies.getAll();
        },
        setAll(cookiesToSet) {
          cookiesToSet.forEach(({ name, value }) =>
            request.cookies.set(name, value)
          );
          supabaseResponse = NextResponse.next({ request });
          cookiesToSet.forEach(({ name, value, options }) =>
            supabaseResponse.cookies.set(name, value, options)
          );
        },
      },
    }
  );

  const { data: { user } } = await supabase.auth.getUser();

  // Védett route-ok
  const url = request.nextUrl.clone();
  const protectedPaths = ["/creator", "/brand", "/admin"];
  const authPaths = ["/login", "/register"];

  if (!user && protectedPaths.some((p) => url.pathname.startsWith(p))) {
    url.pathname = "/login";
    return NextResponse.redirect(url);
  }

  if (user && authPaths.some((p) => url.pathname.startsWith(p))) {
    url.pathname = "/dashboard";
    return NextResponse.redirect(url);
  }

  return supabaseResponse;
}
```

`middleware.ts` (root):

```
import { updateSession } from "@lib/supabase/middleware";
import type { NextRequest } from "next/server";

export async function middleware(request: NextRequest) {
  return await updateSession(request);
}

export const config = {
  matcher: [
    "/(?!_next/static|_next/image|favicon.ico|api|.*\\.?(?!(svg|png|jpg|jpeg|gif|webp)$).*)",
  ],
};
```

2.6 Auth oldalak — Register

`app/(auth)/register/page.tsx` :

- Két lépéses flow:
- **1. lépés:** Role választás (Creator / Brand) — két nagy kártya
- **2. lépés:** Email + jelszó + magyar GDPR checkbox
- Magic link is opcionális (Supabase)
- Successful regisztráció után átirányítás `/onboarding/[role]` -ra

2.7 Auth oldalak — Login

`app/(auth)/login/page.tsx` :

- Email + jelszó
- "Magic link küldése" opció
- Successful login után átirányítás:
- Ha role=creator → `/creator`
- Ha role=brand → `/brand`
- Ha role=admin → `/admin`

2.8 Tesztelési instrukciók

A Claude Code futtassa:

```
npx tsc --noEmit
npm run dev
```

Tesztelési forgatókönyv:

1. Nyisd meg `http://localhost:3000/register`
2. Válaszd a "Creator" kártyát
3. Add meg egy email-t (pl. `test-creator@example.com`) és jelszót
4. Várj a Supabase Auth megerősítő email-jére
5. Ellenőrizd a Supabase Dashboard → Authentication → Users tab-ban a usert
6. Manuálisan futtass egy SQL query-t:

```
sql SELECT * FROM users; SELECT * FROM creator_profiles;
```

Mindkettőben legyen sor.

👉 2. FÁZIS ELLENŐRZÉSI PONT

- [] Drizzle schema létrejött, migration sikeres
- [] Supabase-ben látszanak a táblák (users, creator_profiles, ads, stb.)
- [] Register oldal működik mindkét role-lal
- [] Login oldal működik
- [] Middleware védi a `/creator`, `/brand`, `/admin` route-okat
- [] Logout működik
- [] Új user létrejöttkor a `users` és megfelelő profile tábla is feltöltődik

Ha mind ✓: "Mehet a 3. fázis"

3. FÁZIS — CREATOR PROFIL RENDSZER

 **Aktor:** Claude Code

 **Időbecslés:** 6-8 óra

 **Cél:** Creator regisztráció, onboarding wizard, profil szerkesztő, portfolio feltöltés, rate card

3.1 Supabase Storage bucket beállítása

A Supabase Dashboard-on:

1. Menj a Storage menüpontra

2. Hozz létre 3 bucket-et:

- **avatars** (public read)
- **banners** (public read)
- **portfolio** (public read)

Vagy SQL-lel (Supabase SQL Editor):

```
INSERT INTO storage.buckets (id, name, public) VALUES
  ('avatars', 'avatars', true),
  ('banners', 'banners', true),
  ('portfolio', 'portfolio', true)
ON CONFLICT (id) DO NOTHING;

-- RLS Policies for avatars
CREATE POLICY "Public read avatars"
  ON storage.objects FOR SELECT
  USING (bucket_id = 'avatars');

CREATE POLICY "Authenticated upload avatars"
  ON storage.objects FOR INSERT
  WITH CHECK (bucket_id = 'avatars' AND auth.role() = 'authenticated');

CREATE POLICY "Users update own avatars"
  ON storage.objects FOR UPDATE
  USING (bucket_id = 'avatars' AND auth.uid()::text = (storage.foldername(name))[1]);

CREATE POLICY "Users delete own avatars"
  ON storage.objects FOR DELETE
  USING (bucket_id = 'avatars' AND auth.uid()::text = (storage.foldername(name))[1]);

-- Ugyanezek a policyk a banners és portfolio bucketekhez is
```

A Claude Code másolja ezt és minden bucketre futtassa le.

3.2 Creator onboarding wizard

`app/(auth)/onboarding/creator/page.tsx` — 4 lépéses wizard:

1. lépés: Alapadatok

- Username (egyedi, kebab-case automatikus)

- Megjelenített név
- Bio (max 500 karakter, élő számláló)
- Lokáció (város + megye dropdown)
- Kor (opcionális)
- Nem (opcionális, "férfi/nő/egyéb/nem mondom meg")

2. lépés: Kategória és nyelvek

- Stílus/kategória chip-választó (max 3, a `CREATOR_CATEGORIES` listából)
- Beszélt nyelvek multi-select

3. lépés: Social linkek + manuális follower

- Instagram URL + manuálisan megadott követőszám
- TikTok URL + manuálisan megadott követőszám
- Facebook URL + manuálisan megadott követőszám (opcionális)
- YouTube URL + manuálisan megadott feliratkozók (opcionális)
- **Megjegyzés szöveg:** "A követőszámokat heti automatikus ellenőrzéssel frissítjük. A V2-ben hivatalos OAuth verifikáció érkezik."

4. lépés: Első portfolio + rate card

- Min. 1 portfolio elem feltöltése (video vagy kép, opcionálisan)
- Rate card legalább 1 szolgáltatással
- "Profil mentése" gomb

A wizard végén: `creator_profiles` és `portfolio_items` táblák update-elve, redirect `/creator/dashboard`-ra.

Claude Code: használj `react-hook-form` + `zod` validációt minden lépéshez, állapot localStorage-ban (a wizard-on belül, nem külső adat).

3.3 Creator dashboard layout

`app/(dashboard)/creator/layout.tsx` :

```
// Sidebar navigáció
const navItems = [
  { href: "/creator", label: "Áttekintés", icon: LayoutDashboard },
  { href: "/creator/profile", label: "Profil szerkesztése", icon: User },
  { href: "/creator/portfolio", label: "Portfolio", icon: ImageIcon },
  { href: "/creator/applications", label: "Pályázataim", icon: FileText },
  { href: "/creator/ads", label: "Hirdetések", icon: Megaphone },
  { href: "/creator/messages", label: "Üzenetek", icon: MessageSquare },
  { href: "/creator/reviews", label: "Értékelések", icon: Star },
  { href: "/creator/subscription", label: "Előfizetés", icon: CreditCard },
  { href: "/creator/settings", label: "Beállítások", icon: Settings },
];
```

3.4 Profil szerkesztő oldal

`app/(dashboard)/creator/profile/page.tsx` :

- Tabokra bontva (shadcn Tabs):

- **Alapadatok** — név, bio, lokáció, kor, nem, kategóriák, nyelvek
- **Megjelenés** — avatar feltöltés, banner feltöltés
- **Eszközök** — phone, camera, microphone, editing
- **Social fiókok** — Instagram, TikTok, Facebook, YouTube URL-ek és követőszámok
- **Rate card** — szolgáltatás+ár pár add/remove/edit

Minden tab külön Server Action-nel ment.

3.5 Portfolio kezelő

`app/(dashboard)/creator/portfolio/page.tsx` :

- Drag-and-drop sorrendezés (`@dnd-kit/sortable`)
- Új elem feltöltés modal:
- Type: Video vagy Photo
- Fájl feltöltés Supabase Storage-ba
- Title (opcionális)
- Description (opcionális)
- Kategória chip-ek
- Max 15 elem
- Video preview thumbnail automatikus generálás (canvas-szal kliens oldali)

Csomag installálás:

```
npm install @dnd-kit/core @dnd-kit/sortable @dnd-kit/utilities
```

3.6 Rate card szerkesztő

A profil szerkesztőn belül, dinamikus form:

```
type RateCardItem = {  
  service: string;           // pl. "30s UGC videó"  
  priceHuf: number;          // pl. 25000  
  description?: string;      // opcionális  
};
```

UI:

- "Új szolgáltatás" gomb
- Minden sor: input (szolgáltatás neve), input (ár Ft-ban), opcionális leírás textarea, törlés gomb
- Drag handle az átrendehez

3.7 Server Actions

`app/actions/creator-profile.ts` :

```

"use server";

import { z } from "zod";
import { createClient } from "@lib/supabase/server";
import { db } from "@lib/db";
import { creatorProfiles } from "@lib/db/schema";
import { eq } from "drizzle-orm";
import { revalidatePath } from "next/cache";

const updateProfileSchema = z.object({
  displayName: z.string().min(2).max(100),
  bio: z.string().max(500).optional(),
  city: z.string().max(100).optional(),
  county: z.string().max(50).optional(),
  age: z.number().int().min(13).max(100).optional(),
  gender: z.string().max(20).optional(),
  categories: z.array(z.string()).max(3),
  languages: z.array(z.string()).min(1),
});

export async function updateCreatorProfile(input: z.infer<typeof updateProfileSchema>) {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();

  if (!user) {
    return { error: "Nincs bejelentkezve" };
  }

  const validated = updateProfileSchema.safeParse(input);
  if (!validated.success) {
    return { error: "Érvénytelen adatok" };
  }

  await db.update(creatorProfiles)
    .set({ ...validated.data, updatedAt: new Date() })
    .where(eq(creatorProfiles.userId, user.id));

  revalidatePath("/creator/profile");
  return { success: true };
}

```

Hasonló minden további művelethez (portfolio, rate card, social linkek).

3.8 Username generálás

`lib/utils/username.ts`:

```

export function generateUsername(displayName: string): string {
  return displayName
    .toLowerCase()
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "") // ékezetek levétele
    .replace(/^[a-z0-9]+/g, "-")
    .replace(/^-+|-$/g, "")
    .substring(0, 50);
}

```

A regisztráció során generálja a default username-et, ellenőrizze egyediségét, ha foglalt akkor szám-suffix.

3. FÁZIS ELLENŐRZÉSI PONT

- [] Új creator regisztráció után onboarding wizard végigvezet 4 lépésen
- [] Profil szerkesztő minden mezőt mentés gomb után perzisztál
- [] Avatar és banner feltöltés Supabase Storage-ba működik
- [] Portfolio feltöltés (video és kép) működik
- [] Drag-and-drop sorrendezés portfólióban működik
- [] Rate card hozzáadás/törlés/szerkesztés működik
- [] Username egyedi, ékezetek nélküli
- [] Nincs TypeScript hiba

Ha mind ✓: "Mehet a 4. fázis"



4. FÁZIS — BRAND OLDAL + BÖNGÉSZÉS



Aktor: Claude Code



Időbecslés: 6-8 óra



Cél: Brand regisztráció, creator browse oldal, részletes profil oldal, üzenetküldés

4.1 Brand onboarding

`app/(auth)/onboarding/brand/page.tsx` :

1 lépéses egyszerű form (brand-nek nem kell sok adat indulásnál):

- Cégnév (kötelező)
- Weboldal URL (opcionális)
- Kapcsolattartó név (opcionális)
- Iparág (dropdown — opcionális)

Részletes adatok (adószám, székhely, telefon) **csak az első hirdetésfeladásnál vagy kapcsolatfelvételnél** kerülnek elkérésre.

4.2 Brand dashboard layout

`app/(dashboard)/brand/layout.tsx` :

```
const navItems = [
  { href: "/brand", label: "Áttekintés", icon: LayoutDashboard },
  { href: "/brand/browse", label: "Creatorok böngészése", icon: Search },
  { href: "/brand/saved", label: "Mentett creatorok", icon: Heart },
  { href: "/brand/ads", label: "Hirdetéseim", icon: Megaphone },
  { href: "/brand/messages", label: "Üzenetek", icon: MessageSquare },
  { href: "/brand/profile", label: "Cég profil", icon: Building2 },
];
```

4.3 Creator browse oldal — A LISTANÉZET

`app/(public)/creators/page.tsx` — **publikus, bárki elérheti** (NEM csak brand-eknek)

Layout:

- Header: keresőmező + filter gomb (mobil)
- Bal oldali sticky sidebar (desktop) — szűrők
- Középső grid — creator card-ok, 3 oszlop desktop, 1 mobil
- Felül rendezés dropdown

Szűrők (sidebar):

```
type CreatorFilters = {
  search?: string;
  categories?: string[];
  city?: string;
  county?: string;
  minAge?: number;
  maxAge?: number;
  gender?: "male" | "female" | "any";
  minPriceHuf?: number;
  maxPriceHuf?: number;
  languages?: string[];
  minInstagramFollowers?: number;
  minTiktokFollowers?: number;
  verifiedOnly?: boolean;
  minRating?: number;
};
```

Rendezés opciók:

- Kiemelt (default) — featured + rating + last activity
- Legújabb
- Legjobb értékelés
- Olcsóbb-Drágább
- Drágább-Olcsóbb

URL paraméterek — minden szűrő szinkronizálva legyen URL search params-ba, hogy megosztható legyen a link.

4.4 Creator card komponens

`components/creator/CreatorCard.tsx` :

```

type Props = {
  creator: {
    id: string;
    username: string;
    displayName: string;
    avatarUrl: string | null;
    city: string | null;
    categories: string[];
    instagramFollowers: number | null;
    instagramVerified: boolean;
    tiktokFollowers: number | null;
    tiktokVerified: boolean;
    isFeatured: boolean;
    averageRating: string | null;
    reviewCount: number;
  };
};

export function CreatorCard({ creator }: Props) {
  return (
    <Link
      href={`\creators/${creator.username}`}
      className={cn(
        "group block rounded-xl border bg-card p-4 transition-all hover:shadow-md",
        creator.isFeatured && "ring-2 ring-accent lime-glow"
      )}
    >
      {/\* Header: avatar + rating \*/}
      <div className="flex items-start justify-between">
        <Avatar className="h-16 w-16">
          <AvatarImage src={creator.avatarUrl ?? undefined} /\>
          <AvatarFallback>{creator.displayName.charAt(0)}</AvatarFallback>
        </Avatar>
        {creator.averageRating && (
          <div className="flex items-center gap-1 text-sm">
            <Star className="h-4 w-4 fill-accent text-accent" /\>
            <span className="font-medium">{creator.averageRating}</span>
            <span className="text-muted-foreground">({creator.reviewCount})</span>
          </div>
        )}
      </div>

      {/\* Featured badge \*/}
      {creator.isFeatured && (
        <Badge className="mt-2 bg-accent text-accent-foreground">
          ★ Kiemelt
        </Badge>
      )}

      {/\* Név + lokáció \*/}
      <h3 className="mt-3 text-lg font-semibold">{creator.displayName}</h3>
      {creator.city && (
        <p className="text-sm text-muted-foreground">📍 {creator.city}</p>
      )}

      {/\* Kategóriák \*/}
      <div className="mt-2 flex flex-wrap gap-1">
        <span className="text-sm font-medium">Stílus:</span>
        <span className="text-sm">
          {creator.categories.map(c => CREATOR_CATEGORIES.find(cat => cat.value === c)?.label).join(" · ")}
        </span>
      </div>

      {/\* Social \*/}

```

```

<div className="mt-3 space-y-1 text-sm">
  {creator.instagramFollowers && (
    <div>
      <span className="font-medium">IG</span> Instagram:{" "}
      <span className="font-semibold">{creator.instagramFollowers.toLocaleString("hu-HU")}
    </span>{" "}
      követő {creator.instagramVerified && "✓"}
    </div>
  )}
  {creator.tiktokFollowers && (
    <div>
      <span className="font-medium">TT</span> TikTok:{" "}
      <span className="font-semibold">{creator.tiktokFollowers.toLocaleString("hu-HU")}</
    span>{" "}
      követő {creator.tiktokVerified && "✓"}
    </div>
  )}
</div>

{/* CTA */}
<div className="mt-4 text-sm font-semibold text-accent group-hover:underline">
  Profil megtekintése →
</div>
</Link>
);
}

```

4.5 Creator részletes profil oldal

`app/(public)/creators/[username]/page.tsx` :

7 szekcióra bontva (a spec szerint):

1. Hero

- Banner kép (vagy gradient fallback)
- Profilkép (kerek 120px)
- Név
- Lokáció
- ★ rating
- Featured badge
- "Üzenetet küldök" CTA (brand-nek, nem creator-nek)

2. Bemutakozás

- Bio (max 500 karakter)
- Stílus chip-ek
- Beszélt nyelvek
- Eszközök kártyákban

3. Social Stats

- IG / TT / FB / YT (külön sorok)
- Követőszám + verifikált ✓ pipa (ha igen)
- "Frissítve: 2 órája" timestamp
- Kattintható, új tabban nyílik

4. Portfolio

- Tab-ok: Videók / Fotók / TikTok beágyazás / Instagram beágyazás
- Grid (3 oszlop desktop)
- Lightbox (`yet-another-react-lightbox` csomag)
- oEmbed integráció TikTok és Instagram-hoz

```
npm install yet-another-react-lightbox
```

5. Rate card

- Lista: szolgáltatás → ár (Ft)
- "Egyedi árajánlat kérése" CTA gomb

6. Értékelések

- Csillag-átlag big number
- Distribution chart (5★, 4★, 3★ stb.)
- Review-k listája — szűrhető csillagok szerint
- Creator válaszai megjelennek

7. Hasonló creatorok

- Alul, ugyanabból a kategóriából 3-4 ajánlás

4.6 oEmbed integráció

`lib/utils/oembed.ts` :

```
export async function getTikTokEmbed(url: string) {
  const response = await fetch(
    `https://www.tiktok.com/oembed?url=${encodeURIComponent(url)}`
  );
  if (!response.ok) return null;
  return response.json();
}

export async function getInstagramEmbed(url: string) {
  const accessToken = process.env.META_GRAPH_API_TOKEN;
  if (!accessToken) return null;

  const response = await fetch(
    `https://graph.facebook.com/v18.0/instagram_oembed?url=${encodeURIComponent(url)}&access_token=${accessToken}`
  );
  if (!response.ok) return null;
  return response.json();
}
```

Megjegyzés: Instagram oEmbed-hez Facebook Developer fiók és Meta Graph API token kell. **V2-ben** csináljuk meg — most TikTok-ot priorizáljuk, Instagram-hoz egyszerű URL-megjelenítés.

4.7 Üzenetküldés brand → creator

`components/brand/SendMessageModal.tsx` :

Modal a "Üzenetet küldök" gombra:

- Tárgy (opcionális, max 200 char)
- Üzenet (kötelező, max 5000 char)
- Költségvetés-tartomány (opcionális, két szám HUF-ban)
- "Küldés" gomb

Server Action:

```
"use server";

export async function sendMessage(input: {
  toUsername: string;
  subject?: string;
  body: string;
  budgetHint?: number;
}) {
  // 1. Authentikáció ellenőrzés (csak brand küldhet)
  // 2. Receiver user megkeresése username alapján
  // 3. messages táblába insert
  // 4. Resend-en keresztül email a creator-nek
  // 5. notifications táblába insert
}
```

4.8 Mentett creatorok

Új tábla a schema-ba:

```
export const savedCreators = pgTable("saved_creators", {
  brandId: uuid("brand_id").notNull().references(() => brandProfiles.id, { onDelete: "cascade" })
,
  creatorId: uuid("creator_id").notNull().references(() => creatorProfiles.id, { onDelete: "cascade" }),
  createdAt: timestamp("created_at").notNull().defaultNow(),
  notes: text("notes"),
}, (table) => ({
  pk: primaryKey({ columns: [table.brandId, table.creatorId] }),
}));
```

Generálj új migration-t és futtasd.

4.9 Brand profil oldal

`app/(dashboard)/brand/profile/page.tsx`:

- Cégnév szerkesztése
- Weboldal URL
- Logo feltöltés (Supabase Storage `logos` bucket)
- Kapcsolattartó adatok
- **Részletes adatok** szekció (adószám, székhely, telefon) — opcionális, "csak akkor kell, ha hirdetést akar feladni" felirattal



4. FÁZIS ELLENŐRZÉSI PONT

- [] Brand regisztráció és onboarding működik

- [] Brand dashboard sidebar minden menüpontja működik
- [] Creator browse oldal publikusan elérhető
- [] Minden szűrő működik (kategória, lokáció, ár, követőszám, stb.)
- [] Szűrők URL-ben szinkronizálva (megosztható linkek)
- [] Creator card design responsive (3 col desktop, 1 col mobil)
- [] Featured creatorok kiemelten jelennek meg arany kerettel
- [] Creator részletes profil 7 szekciója renderelődik
- [] oEmbed TikTok-hoz működik
- [] Üzenetküldés brand → creator működik (email + DB record)
- [] Mentett creatorok funkció működik (heart ikon toggle)

Ha mind ✓: "Mehet az 5. fázis"

5. FÁZIS — HIRDETÉSRENDSZER + PÁLYÁZÁS

 **Aktor:** Claude Code

 **Időbecslés:** 6-8 óra

 **Cél:** Brand hirdetést tud feladni, creator pályázhat, brand elfogadhat/elutasíthat, collaboration record létrejön

5.1 Hirdetés feladás form

`app/(dashboard)/brand/ads/new/page.tsx` — multi-step form (vagy egy hosszú scrollable):

Mezők:

```
const adFormSchema = z.object({
  title: z.string().min(5).max(80),
  description: z.string().min(50).max(2000),
  categories: z.array(z.string()).min(1).max(3),
  contentType: z.enum(["video", "photo", "both"]),
  itemCount: z.number().int().min(1).max(20),
  budgetMinHuf: z.number().int().min(1000),
  budgetMaxHuf: z.number().int().min(1000),
  deadline: z.date().min(new Date()),
  location: z.string().max(200).optional(),
  usageRights: z.enum(["organic", "paid_ads", "perpetual"]),
  referenceLinks: z.array(z.string().url()).max(5),
}).refine((data) => data.budgetMaxHuf >= data.budgetMinHuf, {
  message: "A maximum költségvetés nem lehet kisebb a minimumnál",
  path: ["budgetMaxHuf"],
});
```

UI elemek:

- **Cím** — text input, élő karakter számláló
- **Részletes leírás** — textarea, markdown támogatás opcionális
- **Kategóriák** — chip-multi-select (max 3)
- **Tartalom típus** — radio (Videó / Fotó / Mindkettő)
- **Darabszám** — number input (1-20)
- **Költségvetés** — két number input (Min / Max) HUF-ban
- **Deadline** — date picker (`react-day-picker` shadcn-ben)
- **Lokáció** — opcionális text input (csak ha helyszín kötött)
- **Usage rights** — radio (Organic / Paid ads / Perpetual)
- **Referencia linkek** — dinamikus list, max 5 URL

Submit flow:

```
async function createAd(input: AdFormInput) {
  // 1. Authentikáció ellenőrzés (csak brand)
  // 2. Brand profil teljes-e? (taxNumber, address kötelező hirdetésfeladásnál)
  // 3. Aktív hirdetések száma < 5
  // 4. Insert ads táblába status="pending"
  // 5. Admin email értesítés (új moderálandó hirdetés)
  // 6. Redirect /brand/ads/[id]-ra
}
```

5.2 Hirdetések listája (brand oldal)

`app/(dashboard)/brand/ads/page.tsx` :

- Táblázat: cím, kategóriák, költségvetés, deadline, status, pályázatok száma, létrehozva
- Status badge színek: pending (sárga), active (zöld), closed (szürke), rejected (piros)
- Sorkikattintás → részletes oldalra

5.3 Hirdetés részletes nézet (brand oldal)

`app/(dashboard)/brand/ads/[id]/page.tsx` :

Layout:

- Felül: hirdetés adatok + status
- Alul: **beérkezett pályázatok listája**

Pályázat kártya:

- Creator avatar + név + link a profilra
- Pályázati üzenet
- Ár-ajánlat HUF-ban
- Beadás dátuma
- Status: pending / accepted / rejected
- Gombok: "Elfogadás" / "Elutasítás" / "Üzenetküldés" / "Profil megtekintése"

Elfogadás workflow:

```
"use server";
export async function acceptApplication(applicationId: string, rejectionReason?: string) {
  // 1. Application státusz update: accepted/rejected
  // 2. Ha accepted: collaboration record létrehozás
  //   - reviewToken generálás (crypto.randomBytes)
  //   - status: "active"
  // 3. Email a creator-nek (accepted vagy rejected)
  // 4. Notification record
  // 5. Ha sok pályázat elfogadva → opcionálisan ad-státusz "closed"
}
```

5.4 Hirdetések publikus feed (creator oldal)

`app/(public)/ads/page.tsx` — bárki láthatja, de pályázni csak bejelentkezett creator tud:

Listanézet:

- Kártya: cím, brand neve+logója (csak ha approved cég), kategóriák, költségvetés, deadline,

content type

- "Pályázom" gomb (csak creator-eknek)

Szűrők (sidebar):

- Kategória
- Költségvetés-tartomány
- Deadline (e héten / e hónapban / mindenki)
- Lokáció
- Content type (videó / fotó / mindkettő)
- Usage rights
- Rendezés: legújabb, deadline közelében, legmagasabb budget

5.5 Pályázás form

`app/(public)/ads/[id]/apply/page.tsx` :

Modal vagy külön oldal:

```
const applicationSchema = z.object({
  message: z.string().min(50).max(2000),
  proposedPriceHuf: z.number().int().min(1000),
});
```

Submit flow:

```
"use server";
export async function createApplication(input) {
  // 1. Auth check (creator)
  // 2. Creator profil teljes-e (avatar, bio, min 1 portfolio elem)
  // 3. Nem pályázott-e már erre? (unique constraint)
  // 4. Daily limit check: max 10 új pályázat/nap
  // 5. Application insert
  // 6. Ad application_count++
  // 7. Email a brand-nek
  // 8. Notification a brand-nek
}
```

5.6 Creator pályázatok lista

`app/(dashboard)/creator/applications/page.tsx` :

Saját pályázatok táblázat:

- Hirdetés címe (kattintható)
- Brand neve
- Pályázat dátuma
- Ár-ajánlat
- Status (pending / accepted / rejected / withdrawn)

Akciók:

- Visszavonás (csak pending státusznál)
- Üzenet a brand-nek (ha accepted)

5.7 Admin moderálás

A pending hirdetések az admin panelon jelennek meg. Részletesen a 8. fázisban — itt csak annyi, hogy:

- Új ad insert után email küldés `ADMIN_EMAIL`-re
- Admin egy gombbal jóváhagy/elutasít
- Approved → status: "active" + `approvedAt` timestamp
- Rejected → status: "rejected" + `rejectionReason`

5.8 Spam védelem

Rate limiting Server Actions-ben:

```
// lib/utils/rate-limit.ts
const userLimits = new Map<string, { count: number; resetAt: number }>();

export function checkRateLimit(key: string, max: number, windowMs: number) {
  const now = Date.now();
  const record = userLimits.get(key);

  if (!record || now > record.resetAt) {
    userLimits.set(key, { count: 1, resetAt: now + windowMs });
    return { allowed: true, remaining: max - 1 };
  }

  if (record.count >= max) {
    return { allowed: false, remaining: 0, resetAt: record.resetAt };
  }

  record.count++;
  return { allowed: true, remaining: max - record.count };
}
```

Használat:

- Új hirdetés feladás: max 3/óra brand-enként
- Új pályázat: max 10/nap creator-enként
- Üzenet küldés: max 20/óra user-enként

5.9 Email sablonok (Resend)

`lib/resend/templates/`:

- `application-received.tsx` — brand-nek, ha új pályázat
- `application-accepted.tsx` — creator-nek
- `application-rejected.tsx` — creator-nek
- `new-ad-pending.tsx` — admin-nak (moderálás)
- `new-message.tsx` — creator vagy brand-nek

React Email csomag:

```
npm install react-email @react-email/components
```

Mintaszablon `application-accepted.tsx`:

```
import { Html, Body, Container, Text, Button, Hr } from "@react-email/components";

export default function ApplicationAcceptedEmail({
  creatorName,
  brandName,
  adTitle,
}): {
  creatorName: string;
  brandName: string;
  adTitle: string;
}) {
  return (
    <Html>
      <Body style={{ fontFamily: "system-ui, sans-serif", backgroundColor: "#fafafa", padding:
20 }}>
        <Container style={{ maxWidth: 600, margin: "auto", backgroundColor: "white", padding:
30, borderRadius: 12 }}>
          <Text style={{ fontSize: 24, fontWeight: "bold" }}>
            🎉 Elfogadták a pályázatodat!
          </Text>
          <Text>Szia {creatorName}!</Text>
          <Text>
            A <strong>{brandName}</strong> elfogadta a pályázatodat a következő hirdetésre:
          </Text>
          <Text style={{ fontWeight: "bold", fontSize: 18 }}>{adTitle}</Text>
          <Text>Lépj kapcsolatba a márkával, hogy egyeztessétek a részleteket.</Text>
          <Button
            href={`${process.env.NEXT_PUBLIC_APP_URL}/creator/applications`}
            style={{ backgroundColor: "#A3E635", color: "#0A0A0A", padding: "12px 24px", borderRa
dius: 8, fontWeight: "bold" }}
          >
            Pályázatom megtekintése
          </Button>
          <Hr style={{ marginTop: 30 }} />
          <Text style={{ fontSize: 12, color: "#888" }}>
            Üdvözlettel, a Creatorz csapata · creatorz.hu
          </Text>
        </Container>
      </Body>
    </Html>
  );
}
```

👋 5. FÁZIS ELLENŐRZÉSI PONT

- [] Brand felad hirdetést → status: pending (admin emailt kap)
- [] Admin (manuálisan SQL-ben) approve-olja a hirdetést → status: active
- [] Hirdetés megjelenik a publikus `/ads` feedben
- [] Creator pályázik a hirdetésre
- [] Brand-nek megérkezik az értesítő email
- [] Brand elfogadja a pályázatot
- [] Creator-nek megérkezik az elfogadó email
- [] Collaboration record létrejön (későbbi review-hoz)
- [] Rate limit működik (10 pályázat/nap teszt)
- [] Szűrők és rendezés a `/ads` oldalon működnek

Ha mind ✓: "Mehet a 6. fázis"

6. FÁZIS — STRIPE SUBSCRIPTIONS + KIEMELÉS

 **Aktor:** Claude Code

 **Időbecslés:** 4-6 óra

 **Cél:** Creator havi 2 990 Ft előfizetés, 7/30 napos kiemelés vásárlás, Stripe Customer Portal

6.1 Stripe termékek és árak létrehozása

A Stripe Dashboard-on (Test mode):

1. Creator havi előfizetés

Products → **Add product** :

- **Name:** Creator havi előfizetés
- **Description:** Profilod aktívan jelenik meg a Creatorz directoryban
- **Pricing:** Recurring
- Price: **2 990 HUF**
- Billing period: **Monthly**
- Currency: **HUF**

Mentsd el a **Price ID**-t (pl. `price_10AbcXYZ...`).

2. 7 napos kiemelés

Products → **Add product** :

- **Name:** Creatorz Kiemelt — 7 napos
- **Description:** Profilod kiemelten jelenik meg 7 napig a főoldalon és a böngészőben
- **Pricing:** One-time
- Price: **4 990 HUF**
- Currency: **HUF**

Mentsd el a **Price ID**-t.

3. 30 napos kiemelés

- **Name:** Creatorz Kiemelt — 30 napos
- **Pricing:** One-time
- Price: **12 990 HUF**

Mentsd el a **Price ID**-t.

Add hozzá ezeket a `.env.local`-hoz:

```
STRIPE_PRICE_CREATOR_MONTHLY=price_...
STRIPE_PRICE_FEATURE_7DAY=price_...
STRIPE_PRICE_FEATURE_30DAY=price_...
```

6.2 Stripe webhook beállítása

Helyi fejlesztéshez (Stripe CLI):

- Telepítsd a Stripe CLI-t: <https://stripe.com/docs/stripe-cli#install>
- Lépj be: `stripe login`
- Indítsd el a webhook forwardingot:

```
stripe listen --forward-to http://localhost:3000/api/webhooks/stripe
```

A parancs egy webhook secretet ír ki (`whsec_...`). Másold be a `.env.local`-ba:

```
STRIPE_WEBHOOK_SECRET=whsec_...
```

Production-hez (Vercel deployment után):

Stripe Dashboard → `Developers` → `Webhooks` → `Add endpoint` :

- URL: `https://creatorz.hu/api/webhooks/stripe`
- Events:
 - `checkout.session.completed`
 - `customer.subscription.created`
 - `customer.subscription.updated`
 - `customer.subscription.deleted`
 - `invoice.paid`
 - `invoice.payment_failed`

6.3 Stripe client setup

`lib/stripe/client.ts` :

```
import Stripe from "stripe";

export const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!, {
  apiVersion: "2024-12-18.acacia",
  typescript: true,
});
```

6.4 Creator subscription oldal

`app/(dashboard)/creator/subscription/page.tsx` :

Tartalom:

- Aktuális státusz badge (Aktív / Lejárt / Próbaidőszak / Lemondva)
- Következő esedékesség dátuma
- Fizetési előzmények táblázat
- "Előfizetés módosítása" gomb → Stripe Customer Portal
- Ha még nincs subscription:
- Ha az admin toggle szerint **kötelező** a fizetés: "Előfizetés indítása — 2 990 Ft/hó" gomb
- Ha **ingyenes** módban van: "Az ingyenes regisztrációs időszak alatt nincs szükség előfizetésre" üzenet

6.5 Subscribe gomb → Stripe Checkout

Server Action `app/actions/stripe.ts`:

```
"use server";

import { stripe } from "@lib/stripe/client";
import { createClient } from "@lib/supabase/server";
import { db } from "@lib/db";
import { users, subscriptions } from "@lib/db/schema";
import { eq } from "drizzle-orm";
import { redirect } from "next/navigation";

export async function startCreatorSubscription() {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Nincs bejelentkezve");

  // Stripe Customer létrehozása (ha még nincs)
  let dbUser = await db.query.users.findFirst({
    where: eq(users.authId, user.id),
    with: { subscription: true },
  });

  let customerId = dbUser?.subscription?.stripeCustomerId;

  if (!customerId) {
    const customer = await stripe.customers.create({
      email: user.email!,
      metadata: { userId: dbUser!.id },
    });
    customerId = customer.id;
  }

  // Checkout Session
  const session = await stripe.checkout.sessions.create({
    customer: customerId,
    mode: "subscription",
    payment_method_types: ["card"],
    line_items: [
      {
        price: process.env.STRIPE_PRICE_CREATOR_MONTHLY,
        quantity: 1,
      },
    ],
    success_url: `${process.env.NEXT_PUBLIC_APP_URL}/creator/subscription?success=true`,
    cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/creator/subscription?canceled=true`,
    locale: "hu",
    billing_address_collection: "required",
    automatic_tax: { enabled: true },
  });

  redirect(session.url!);
}
```

6.6 Kiemelés vásárlás

Hasonló logikával:

```
export async function purchaseFeature(type: "7day" | "30day") {
  const priceId = type === "7day"
    ? process.env.STRIPE_PRICE_FEATURE_7DAY
    : process.env.STRIPE_PRICE_FEATURE_30DAY;

  // ... user lookup, Stripe customer ...

  const session = await stripe.checkout.sessions.create({
    customer: customerId,
    mode: "payment", // One-time payment!
    payment_method_types: ["card"],
    line_items: [{ price: priceId, quantity: 1 }],
    metadata: {
      type: `feature_${type}`,
      creatorProfileId: creatorProfile.id,
    },
    success_url: `${process.env.NEXT_PUBLIC_APP_URL}/creator?feature=success`,
    cancel_url: `${process.env.NEXT_PUBLIC_APP_URL}/creator?feature=canceled`,
    locale: "hu",
    automatic_tax: { enabled: true },
  });

  redirect(session.url!);
}
```

6.7 Webhook handler

`app/api/webhooks/stripe/route.ts` :

```

import { headers } from "next/headers";
import { NextResponse } from "next/server";
import { stripe } from "@lib/stripe/client";
import { db } from "@lib/db";
import { subscriptions, featurePurchases, creatorProfiles, users } from "@lib/db/schema";
import { eq } from "drizzle-orm";
import type Stripe from "stripe";

export async function POST(req: Request) {
  const body = await req.text();
  const signature = (await headers()).get("stripe-signature")!;

  let event: Stripe.Event;
  try {
    event = stripe.webhooks.constructEvent(
      body,
      signature,
      process.env.STRIPE_WEBHOOK_SECRET!
    );
  } catch (err) {
    console.error("Webhook signature verification failed:", err);
    return new NextResponse("Webhook Error", { status: 400 });
  }

  try {
    switch (event.type) {
      case "checkout.session.completed": {
        const session = event.data.object as Stripe.Checkout.Session;

        if (session.mode === "subscription") {
          // Creator subscription
          const sub = await stripe.subscriptions.retrieve(
            session.subscription as string
          );
          const dbUser = await db.query.users.findFirst({
            where: eq(users.email, session.customer_email!),
          });
          if (!dbUser) throw new Error("User not found");

          await db.insert(subscriptions).values({
            userId: dbUser.id,
            stripeCustomerId: session.customer as string,
            stripeSubscriptionId: sub.id,
            status: sub.status as any,
            currentPeriodStart: new Date(sub.current_period_start * 1000),
            currentPeriodEnd: new Date(sub.current_period_end * 1000),
          }).onConflictDoUpdate({
            target: subscriptions.userId,
            set: {
              stripeSubscriptionId: sub.id,
              status: sub.status as any,
              currentPeriodStart: new Date(sub.current_period_start * 1000),
              currentPeriodEnd: new Date(sub.current_period_end * 1000),
            },
          });
        } else if (session.mode === "payment") {
          // Feature purchase
          const type = session.metadata?.type;
          const creatorProfileId = session.metadata?.creatorProfileId;

          if (type === "feature_7day" || type === "feature_30day") {
            const days = type === "feature_7day" ? 7 : 30;
            const startsAt = new Date();
            const endsAt = new Date(startsAt.getTime() + days * 24 * 60 * 60 * 1000);
          }
        }
      }
    }
  }
}

```

```

    await db.insert(featurePurchases).values({
      creatorId: creatorProfileId!,
      type: type === "feature_7day" ? "7day" : "30day",
      stripePaymentIntentId: session.payment_intent as string,
      amountHuf: type === "feature_7day" ? 4990 : 12990,
      startsAt,
      endsAt,
    });

    // Update creator profile is_featured + featured_until
    await db.update(creatorProfiles)
      .set({ isFeatured: true, featuredUntil: endsAt })
      .where(eq(creatorProfiles.id, creatorProfileId!));
  }
}
break;
}

case "customer.subscription.updated":
case "customer.subscription.deleted": {
  const sub = event.data.object as Stripe.Subscription;
  await db.update(subscriptions)
    .set({
      status: sub.status as any,
      currentPeriodEnd: new Date(sub.current_period_end * 1000),
      cancelAtPeriodEnd: sub.cancel_at_period_end,
      canceledAt: sub.canceled_at ? new Date(sub.canceled_at * 1000) : null,
      updatedAt: new Date(),
    })
    .where(eq(subscriptions.stripeSubscriptionId, sub.id));
  break;
}

case "invoice.payment_failed": {
  // Email a creator-nek a sikertelen fizetésről
  // Status update past_due-ra
  break;
}
}

return NextResponse.json({ received: true });
} catch (err) {
  console.error("Webhook handler error:", err);
  return new NextResponse("Webhook Handler Error", { status: 500 });
}
}

```

6.8 Stripe Customer Portal

A creatorok a saját előfizetésüket itt tudják kezelni:

```
"use server";

export async function openCustomerPortal() {
  const supabase = await createClient();
  const { data: { user } } = await supabase.auth.getUser();
  if (!user) throw new Error("Nincs bejelentkezve");

  const dbUser = await db.query.users.findFirst({
    where: eq(users.authId, user.id),
    with: { subscription: true },
  });

  if (!dbUser?.subscription?.stripeCustomerId) {
    throw new Error("Nincs előfizetés");
  }

  const session = await stripe.billingPortal.sessions.create({
    customer: dbUser.subscription.stripeCustomerId,
    return_url: `${process.env.NEXT_PUBLIC_APP_URL}/creator/subscription`,
    locale: "hu",
  });

  redirect(session.url);
}
```

6.9 Cron — Lejárt featured profilok visszaállítása

`app/api/cron/expire-featured/route.ts`:

```
import { db } from "@lib/db";
import { creatorProfiles } from "@lib/db/schema";
import { and, eq, lte, isNotNull } from "drizzle-orm";

export async function GET(req: Request) {
  const authHeader = req.headers.get("authorization");
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return new Response("Unauthorized", { status: 401 });
  }

  // Reset is_featured where featured_until has passed
  const now = new Date();
  const result = await db.update(creatorProfiles)
    .set({ isFeatured: false, featuredUntil: null })
    .where(and(
      eq(creatorProfiles.isFeatured, true),
      eq(creatorProfiles.isAdminFeatured, false),
      isNotNull(creatorProfiles.featuredUntil),
      lte(creatorProfiles.featuredUntil, now)
    ));

  return Response.json({ updated: result.length });
}
```

Vercel Cron beállítása `vercel.json`:

```
{
  "crons": [
    {
      "path": "/api/cron/expire-featured",
      "schedule": "0 */6 * * *"
    },
    {
      "path": "/api/cron/scrape-followers",
      "schedule": "0 3 * * 1"
    },
    {
      "path": "/api/cron/review-emails",
      "schedule": "0 10 * * *"
    }
  ]
}
```

👉 6. FÁZIS ELLENŐRZÉSI PONT

- [] Creator subscription Test mode-ban végigmegy (4242 4242 4242 4242 test kártya)
- [] Subscription rekord létrejön a DB-ben
- [] 7 napos kiemelés vásárlás → creator `isFeatured=true` és kiemelve a listán
- [] 30 napos kiemelés ugyanígy működik
- [] Webhook fogadja az eseményeket (stripe listen logokban látszik)
- [] Customer Portal megnyílik magyar nyelven
- [] Cron endpoint manuálisan tesztelve működik

Ha mind ✓: "Mehet a 7. fázis"

★ 7. FÁZIS — REVIEW RENDSZER (MODELL A)

 **Aktor:** Claude Code

 **Időbecslés:** 3-4 óra

 **Cél:** Brand review-t írhat creator-re az elfogadott pályázat után, 7 nappal automatikus email, creator válaszolhat

7.1 Cron — 7 napos review email

`app/api/cron/review-emails/route.ts :`

```

import { db } from "@lib/db";
import { collaborations, brandProfiles, creatorProfiles, users } from "@lib/db/schema";
import { and, eq, lte, isNull } from "drizzle-orm";
import { resend } from "@lib/resend/client";
import ReviewPromptEmail from "@lib/resend/templates/review-prompt";
import crypto from "crypto";

export async function GET(req: Request) {
  const authHeader = req.headers.get("authorization");
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return new Response("Unauthorized", { status: 401 });
  }

  // Collaborations elder than 7 days, no review email sent yet
  const sevenDaysAgo = new Date(Date.now() - 7 * 24 * 60 * 60 * 1000);

  const pending = await db.query.collaborations.findMany({
    where: and(
      lte(collaborations.acceptedAt, sevenDaysAgo),
      isNull(collaborations.reviewEmailSentAt),
      eq(collaborations.status, "active")
    ),
    with: {
      brand: { with: { user: true } },
      creator: true,
    },
  });

  let sent = 0;
  for (const collab of pending) {
    // Generate review token
    const reviewToken = crypto.randomBytes(32).toString("hex");

    // Update collab
    await db.update(collaborations)
      .set({
        reviewToken,
        reviewEmailSentAt: new Date(),
        status: "review_pending",
      })
      .where(eq(collaborations.id, collab.id));

    // Send email
    await resend.emails.send({
      from: process.env.EMAIL_FROM!,
      to: collab.brand.user.email,
      subject: `Hogyan ment a közös munka ${collab.creator.displayName}-rel?`,
      react: ReviewPromptEmail({
        brandName: collab.brand.companyName,
        creatorName: collab.creator.displayName,
        reviewUrl: `${process.env.NEXT_PUBLIC_APP_URL}/review/${reviewToken}`,
      }),
    });

    sent++;
  }

  return Response.json({ sent });
}

```

7.2 Review beküldési oldal (token-alapú)

`app/review/[token]/page.tsx` — **publikus, login nélkül is megnyitható:**

- Token alapján lekérdezés: `collaborations` táblából
- Ha valid és status="review_pending":
- Megjelenít: brand neve, creator neve, "Hogyan ment?"
- Form: 4 csillagos rating (overall + comm + quality + deadline) + text
- "Beküldés" gomb

```
const reviewSchema = z.object({
  overallRating: z.number().int().min(1).max(5),
  communicationRating: z.number().int().min(1).max(5),
  qualityRating: z.number().int().min(1).max(5),
  deadlineRating: z.number().int().min(1).max(5),
  text: z.string().min(30).max(2000),
});
```

7.3 Review beküldés Server Action

```
"use server";
export async function submitReview(token: string, input: z.infer<typeof reviewSchema>) {
  // 1. Token validáció
  const collab = await db.query.collaborations.findFirst({
    where: eq(collaborations.reviewToken, token),
    with: { creator: true, brand: true },
  });
  if (!collab || collab.status !== "review_pending") {
    return { error: "Érvénytelen vagy lejárt link" };
  }

  // 2. Review insert
  const editedUntil = new Date(Date.now() + 24 * 60 * 60 * 1000); // 24 óra
  await db.insert(reviews).values({
    collaborationId: collab.id,
    brandId: collab.brandId,
    creatorId: collab.creatorId,
    overallRating: input.overallRating,
    communicationRating: input.communicationRating,
    qualityRating: input.qualityRating,
    deadlineRating: input.deadlineRating,
    text: input.text,
    editedUntil,
  });

  // 3. Collab status update
  await db.update(collaborations)
    .set({ status: "reviewed" })
    .where(eq(collaborations.id, collab.id));

  // 4. Creator profile review count + averageRating recalc
  await recalculateCreatorRating(collab.creatorId);

  // 5. Email a creator-nek ("Új értékelést kaptál")
  // 6. Notification

  return { success: true };
}

async function recalculateCreatorRating(creatorId: string) {
  const allReviews = await db.query.reviews.findMany({
    where: and(
      eq(reviews.creatorId, creatorId),
      eq(reviews.hidden, false)
    ),
  });

  const count = allReviews.length;
  const avg = count > 0
    ? allReviews.reduce((sum, r) => sum + r.overallRating, 0) / count
    : null;

  await db.update(creatorProfiles)
    .set({
      reviewCount: count,
      averageRating: avg ? avg.toFixed(2) : null,
    })
    .where(eq(creatorProfiles.id, creatorId));
}
```

7.4 Review komponensek

`components/shared/ReviewCard.tsx` :

```
export function ReviewCard({ review }: { review: ReviewWithRelations }) {
  return (
    <div className="rounded-xl border p-4">
      <div className="flex items-start justify-between">
        <div className="flex items-center gap-3">
          {review.brand.logoUrl && (
            <Avatar><AvatarImage src={review.brand.logoUrl} /></Avatar>
          )}
        <div>
          <p className="font-semibold">{review.brand.companyName}</p>
          <p className="text-sm text-muted-foreground">
            {format(review.createdAt, "yyyy. MMM dd.", { locale: hu })}
          </p>
        </div>
      </div>
      <RatingStars rating={review.overallRating} />
    </div>
    <p className="mt-3 text-sm">{review.text}</p>

    {/* Részletes rating-ek */}
    <div className="mt-3 flex gap-4 text-xs text-muted-foreground">
      <span>Komm.: {review.communicationRating}/5</span>
      <span>Minőség: {review.qualityRating}/5</span>
      <span>Határidő: {review.deadlineRating}/5</span>
    </div>

    {/* Creator válasz */}
    {review.response && (
      <div className="mt-4 rounded-lg bg-muted p-3">
        <p className="text-xs font-semibold text-muted-foreground">Creator válasza:</p>
        <p className="mt-1 text-sm">{review.response.text}</p>
      </div>
    )}
  </div>
);
}
```

7.5 Creator válasz

`app/(dashboard)/creator/reviews/page.tsx` :

- Creator látja az összes review-ját
- "Válaszolok" gomb minden review-n (csak ha még nincs response)
- Modal: textarea (max 500 char) + "Beküldés"
- Beküldés után 24 órás edit window

7.6 Rating distribution chart

`components/shared/RatingDistribution.tsx` :

```

type Props = { reviews: Array<{ overallRating: number }> };

export function RatingDistribution({ reviews }: Props) {
  const counts = [1, 2, 3, 4, 5].map((star) => ({
    star,
    count: reviews.filter((r) => r.overallRating === star).length,
  }));
  const total = reviews.length;

  return (
    <div className="space-y-1">
      {counts.reverse().map(({ star, count }) => {
        const pct = total > 0 ? (count / total) * 100 : 0;
        return (
          <div key={star} className="flex items-center gap-2 text-sm">
            <span className="w-6">{star}★</span>
            <div className="h-2 flex-1 rounded-full bg-muted">
              <div className="h-2 rounded-full bg-accent" style={{ width: `${pct}%` }} />
            </div>
            <span className="w-8 text-right text-xs text-muted-foreground">{count}</span>
          </div>
        );
      })}
    </div>
  );
}

```

7.7 Riport (report) funkcionalitás

- "Bejelentem" gomb minden review-n
- Modal: indok (textarea)
- Submit → `reviews.reported=true`, admin email értesítés
- Admin a panelen átnézi és vagy "elrejt" vagy "visszaállítja"



7. FÁZIS ELLENŐRZÉSI PONT

- [] Egy 7+ napos collaboration esetén a cron generálja a review email-t
- [] Email-ben kapott linkkel a brand kitölti a review-t login nélkül
- [] Review megjelenik a creator profilon
- [] Creator átlag rating helyesen számolódik
- [] Creator válaszolhat egy review-ra
- [] Riport funkció működik (admin notification)
- [] 24 órás edit window aktív
- [] Distribution chart helyes számokat mutat

Ha mind ✓: "Mehet a 8. fázis"



8. FÁZIS — ADMIN PANEL

 **Aktor:** Claude Code

 **Időbecslés:** 4-6 óra

 **Cél:** Teljes admin felület — globális toggle-ek, user-management, moderálás, statisztikák

8.1 Admin role beállítás

Most a Supabase SQL Editor-ben:

```
-- A te user-edet admin-ra promotálok
UPDATE users SET role = 'admin' WHERE email = 'a-te-email-cimed@example.com';
```

Az `.env.local`-ban legyen:

```
ADMIN_EMAIL=a-te-email-cimed@example.com
```

8.2 Admin layout

`app/(dashboard)/admin/layout.tsx`:

```
const navItems = [
  { href: "/admin", label: "Áttekintés", icon: LayoutDashboard },
  { href: "/admin/settings", label: "Beállítások", icon: Settings },
  { href: "/admin/users", label: "Felhasználók", icon: Users },
  { href: "/admin/creators", label: "Creatorok", icon: UserCheck },
  { href: "/admin/brands", label: "Márkák", icon: Building2 },
  { href: "/admin/ads", label: "Hirdetések", icon: Megaphone },
  { href: "/admin/reviews", label: "Értékelések", icon: Star },
  { href: "/admin/reports", label: "Bejelentések", icon: AlertTriangle },
  { href: "/admin/finance", label: "Pénzügy", icon: TrendingUp },
];
```

Middleware ellenőrzés: csak role="admin" user férhet hozzá.

8.3 Admin Settings — globális toggle-ek

`app/(dashboard)/admin/settings/page.tsx`:

A `settings` táblát használjuk key-value store-ként:

```
// Default settings (seed)
const DEFAULT_SETTINGS = {
  creator_subscription_enabled: false,
  creator_subscription_price_huf: 2990,
  feature_7day_price_huf: 4990,
  feature_30day_price_huf: 12990,
  registration_enabled: true,
  auto_approve_creators: false,
  auto_approve_brands: true,
  auto_approve_ads: false,
};
```

UI:

```
<Card>
  <CardHeader>
    <CardTitle>Creator előfizetés</CardTitle>
    <CardDescription>
      Itt szabályozod, hogy a creator regisztráció ingyenes vagy fizetős.
    </CardDescription>
  </CardHeader>
  <CardContent className="space-y-4">
    <div className="flex items-center justify-between">
      <Label>Creator előfizetés kötelező</Label>
      <Switch
        checked={settings.creator_subscription_enabled}
        onCheckedChange={(v) => updateSetting("creator_subscription_enabled", v)}
      />
    </div>
    <div>
      <Label>Havi ár (Ft)</Label>
      <Input
        type="number"
        value={settings.creator_subscription_price_huf}
        onChange={(e) => updateSetting("creator_subscription_price_huf", +e.target.value)}
      />
    </div>
  </CardContent>
</Card>
```

Hasonló kártyák minden setting-hez.

8.4 Users management

`app/(dashboard)/admin/users/page.tsx` :

- Táblázat: email, role, approved, suspended, létrehozva, utolsó belépés
- Szűrők: role, approved, suspended
- Keresés: email
- Akciók sorként:
- Felfüggesztés / visszaállítás
- Role változtatás
- Profil törlés
- Manuális subscription státusz módosítás

8.5 Creators panel

`app/(dashboard)/admin/creators/page.tsx` :

- Lista: avatar, név, username, kategória, regisztrálás dátuma, status, review count, rating
- Új creator-ok várnak jóváhagyásra (`approved=false`) → kiemelten
- Egy klikkes jóváhagyás
- "Admin featured" toggle — független a vásárolt kiemelésről

8.6 Brands panel

Hasonló — cégnév, weboldal, hirdetések száma, aktivitás.

8.7 Ads moderálás

`app/(dashboard)/admin/ads/page.tsx` :

- Status szerinti tabok: Pending / Active / Closed / Rejected
- Pending hirdetések: hirdetés tartalma + "Jóváhagy" / "Elutasít" gombok
- Elutasítás esetén: rejection reason input

8.8 Reports panel

`app/(dashboard)/admin/reports/page.tsx` :

- Bejelentett review-k, profilok, hirdetések
- Részletes nézet: mi a tartalom, ki jelentette, indok
- "Elrejtés" / "Visszaállítás" gombok
- "Beküldő figyelmeztetése" gomb (email)

8.9 Finance panel

`app/(dashboard)/admin/finance/page.tsx` :

- **MRR** (Monthly Recurring Revenue) — élő szám
- Új subscriptionek havonta (chart)
- Lemondott subscriptionek havonta
- Kiemelés-vásárlások havonta + összeg
- Top 10 legtöbbet kereső creator
- Aktív userek 7 / 30 nap
- Új regisztrációk creator vs brand bontásban

Charts: `recharts` csomag

```
npm install recharts
```

8.10 Áttekintés dashboard

`app/(dashboard)/admin/page.tsx` :

- KPI kártyák: össz user, aktív subscription, havi MRR, ma feladott hirdetés

- Gyors linkek: pending moderálás, új report, friss subscription
- Aktivitási feed (legutóbbi 20 esemény)

8. FÁZIS ELLENŐRZÉSI PONT

- [] Admin login után csak az admin user éri el a /admin route-ot
- [] Globális toggle "creator subscription enabled" működik (ki-be kapcsolható)
- [] Settings DB-ben perzisztens
- [] Pending creatorok jóváhagyása működik
- [] Pending hirdetések jóváhagyás/elutasítás működik
- [] User suspended-re állítása letiltja a bejelentkezést
- [] MRR és kpi számok helyesek
- [] Reports panel bejelentett tartalmakat mutatja

Ha mind ✓: "Mehet a 9. fázis"



9. FÁZIS — KÉPGENERÁLÁS + LANDING PAGE



Aktor: Claude Code



Időbecslés: 3-4 óra



Cél: Replicate FLUX kép-generáló integráció, landing page minden szükséges képpel

9.1 Replicate FLUX client

`lib/replicate/client.ts`:

```
import Replicate from "replicate";

export const replicate = new Replicate({
  auth: process.env.REPLICATE_API_TOKEN,
});

// FLUX.1 [schnell] – gyors, olcsó (~$0.003/kép)
// FLUX.1 [dev] – jobb minőség (~$0.025/kép)
// Mi schnell-t használunk

export async function generateImage(prompt: string, opts?: {
  width?: number;
  height?: number;
  aspectRatio?: "1:1" | "16:9" | "9:16" | "4:3" | "3:4";
}) {
  const output = await replicate.run(
    "black-forest-labs/flux-schnell",
    {
      input: {
        prompt,
        aspect_ratio: opts?.aspectRatio || "1:1",
        output_format: "webp",
        output_quality: 90,
        num_outputs: 1,
        go_fast: true,
      },
    }
  );
  // Output is an array of URLs
  return Array.isArray(output) ? output[0] : output;
}
```

Telepítés:

```
npm install replicate
```

9.2 Kép-generáló script

`scripts/generate-images.ts` — automatikusan generálja és lementi az összes szükséges képet:

```

import { generateImage } from "../lib/replicate/client";
import fs from "fs/promises";
import path from "path";

// Image prompts
const IMAGES = [
  // === LANDING PAGE ===
  {
    name: "hero-bg",
    prompt: "Modern minimalist abstract dark background, glowing neon green particles flowing diagonally, deep black gradient, cinematic, 4k quality, atmospheric, digital art, slight green vignette in corners",
    aspectRatio: "16:9" as const,
  },
  {
    name: "feature-creator",
    prompt: "Young confident Hungarian content creator with smartphone filming a TikTok video, modern bright apartment, natural light, soft focus background, lifestyle photography, vibrant colors, professional shot",
    aspectRatio: "4:3" as const,
  },
  {
    name: "feature-brand",
    prompt: "Modern Hungarian e-commerce startup team in a bright office, looking at marketing dashboard on large screen, professional but casual atmosphere, diverse team, soft natural light, professional photography",
    aspectRatio: "4:3" as const,
  },
  {
    name: "feature-collaboration",
    prompt: "Two professionals (one creator with camera, one brand manager with laptop) shaking hands or high-fiving over a coffee table in a bright modern cafe, success moment, candid lifestyle photography",
    aspectRatio: "4:3" as const,
  },
  // === EMPTY STATES ===
  {
    name: "empty-search",
    prompt: "Minimalist line art illustration of a magnifying glass over empty grid, black on white background, simple geometric, modern flat design, vector style, no text",
    aspectRatio: "1:1" as const,
  },
  {
    name: "empty-messages",
    prompt: "Minimalist line art illustration of empty inbox or paper plane, black on white background, simple geometric, modern flat design, vector style, no text",
    aspectRatio: "1:1" as const,
  },
  {
    name: "empty-ads",
    prompt: "Minimalist line art illustration of empty megaphone or broadcast tower, black on white background, simple geometric, modern flat design, vector style, no text",
    aspectRatio: "1:1" as const,
  },
  // === CATEGORY ICONS (16 db) ===
  ...[
    { name: "cat-gasztro", topic: "elegant food plating, chef's hand garnishing dish" },
    { name: "cat-utazas", topic: "travel suitcase with vintage stickers, world map" },
    { name: "cat-divat", topic: "fashion hangers with stylish clothes" },
    { name: "cat-sport", topic: "running shoes and sports gear" },
    { name: "cat-beauty", topic: "makeup brushes and cosmetics" },
    { name: "cat-lifestyle", topic: "coffee cup, notebook, and plant on desk" },
    { name: "cat-tech", topic: "modern smartphone and headphones" },
    { name: "cat-otthon", topic: "modern living room interior" },
    { name: "cat-anyukak", topic: "mother and baby silhouette" },
  ]
];

```

```

    { name: "cat-auto", topic: "sleek modern car silhouette" },
    { name: "cat-fitness", topic: "dumbbells and water bottle" },
    { name: "cat-wellness", topic: "meditation yoga pose silhouette" },
    { name: "cat-vendeglatas", topic: "restaurant table setting" },
    { name: "cat-gyerek", topic: "colorful toys" },
    { name: "cat-allatok", topic: "cute dog and cat silhouettes" },
    { name: "cat-egyeb", topic: "creative box with various items" },
  ].map(c => ({
    name: c.name,
    prompt: `Minimalist flat icon illustration of ${c.topic}, neon lime green (#A3E635) accent
    color on white background, modern flat design, simple geometric shapes, vector style, centered,
    no text`,
    aspectRatio: "1:1" as const,
  })),
  // === DEMO CREATOR AVATARS (10 db, soft launch előtt) ===
  ...Array.from({ length: 10 }, (_, i) => ({
    name: `demo-avatar-${i + 1}`,
    prompt: `Professional portrait photograph of a Hungarian content creator, person ${i + 1}: $
    {
      "young woman with warm smile, casual sweater, natural makeup",
      "young man with beard, hipster style, smiling",
      "middle-aged woman with elegant style, confident expression",
      "young woman with long dark hair, athletic look",
      "young man casual style, urban background",
      "woman in her 30s, business casual, professional smile",
      "young woman blonde hair, lifestyle blogger aesthetic",
      "young man tech enthusiast, glasses, smart casual",
      "woman foodie aesthetic, apron, warm smile",
      "young woman travel blogger, sun-kissed look",
    }[i]}, soft natural lighting, clean studio background, professional headshot, high quality
    photography, candid expression`,
    aspectRatio: "1:1" as const,
  })),
  // === SOCIAL OG IMAGE ===
  {
    name: "og-image",
    prompt: "Modern minimalist banner design, deep black background with neon lime green
    geometric shapes, large text saying 'CREATORZ' in bold sans-serif, subtle particle effects, 16:9
    aspect ratio, premium feel, digital art",
    aspectRatio: "16:9" as const,
  },
];

async function main() {
  const outputDir = path.join(process.cwd(), "public/images/generated");
  await fs.mkdir(outputDir, { recursive: true });

  console.log(`Generating ${IMAGES.length} images...`);

  for (const [i, img] of IMAGES.entries()) {
    console.log(`[${i + 1}/${IMAGES.length}] Generating: ${img.name}`);
    try {
      const url = await generateImage(img.prompt, { aspectRatio: img.aspectRatio });
      const response = await fetch(url as string);
      const buffer = await response.arrayBuffer();
      const filename = `${img.name}.webp`;
      await fs.writeFile(path.join(outputDir, filename), Buffer.from(buffer));
      console.log(` ✓ Saved: ${filename}`);
    } catch (err) {
      console.error(` ✗ Failed: ${img.name}`, err);
    }
    // Rate limit safety
    await new Promise((r) => setTimeout(r, 1000));
  }

  console.log("\n ✓ All images generated!");
}

```

```
}  
  
main().catch(console.error);
```

Futtatás:

```
npx tsx scripts/generate-images.ts
```

Várható futási idő: ~5-10 perc (38 kép)

Várható költség: ~\$0.12 (~50 Ft) — FLUX schnell rendkívül olcsó

9.3 Landing page

app/page.tsx :

Struktúra:

1. Hero szekció

- Sötét háttér + neon lime accent
- H1: "A magyar UGC tartalomgyártók új otthona"
- Alcím: "Találd meg a tökéletes creatort a márkához, vagy regisztrálj creatorként."
- 2 CTA gomb: "Creatorz vagyok →" / "Brandet képviselek →"

2. Hogyan működik (3 lépés)

- "1. Regisztrálj" — ingyenes, 2 perc
- "2. Találd meg a tökéletes párost" — szűrjük neked
- "3. Kezdj el dolgozni" — közvetlen kapcsolatfelvétel

3. Featured creatorok

- Live data: top 6 featured creator
- "Mind megtekintése →" link

4. Két oldal: cégeknek és creatoroknak

- Bal: "Brand vagy?" — előnyök listája + CTA
- Jobb: "Creator vagy?" — előnyök listája + CTA

5. Statisztikák / social proof

- "X aktív creator", "Y elindított kampány", "Z márka regisztrálva"

6. FAQ szekció (collapsible)

- Mennyibe kerül?
- Mi van benne a creator előfizetésben?
- Hogyan védjük a personal adatokat?
- Mi az UGC?

7. Footer

- Linkek: Adatvédelem, ÁSZF, Kapcsolat, GYIK

- Social ikonok (placeholder, később aktiválva)
- Copyright

9.4 Hero design

```
<section className="relative min-h-screen overflow-hidden bg-[#0A0A0A] text-white">
  {/* Background image */}
  <Image
    src="/images/generated/hero-bg.webp"
    alt=""
    fill
    priority
    className="object-cover opacity-40"
  />

  {/* Gradient overlay */}
  <div className="absolute inset-0 bg-gradient-to-b from-transparent to-black/80" />

  {/* Content */}
  <div className="relative z-10 mx-auto flex min-h-screen max-w-6xl flex-col items-center
justify-center px-6 text-center">
    <Badge className="mb-6 bg-accent/20 text-accent border-accent/40">
      ✨ Magyar UGC tartalomgyártók új otthona
    </Badge>

    <h1 className="mb-6 text-balance text-5xl font-bold tracking-tight md:text-7xl">
      Találd meg a tökéletes
      <span className="block text-accent">UGC creatort</span>
      a márkához
    </h1>

    <p className="mb-10 max-w-2xl text-balance text-lg text-white/70 md:text-xl">
      Magyar tartalomgyártók és márkák találkozóhelye. Regisztrálj ingyen,
      böngéssz portfóliókat, vagy add fel a hirdetésed.
    </p>

    <div className="flex flex-col gap-4 sm:flex-row">
      <Button size="lg" asChild className="bg-accent text-accent-foreground hover:bg-accent/90">
        <Link href="/register?role=creator">
          Creatorz vagyok →
        </Link>
      </Button>
      <Button size="lg" variant="outline" asChild className="border-white/40 text-white hover:bg-white/10">
        <Link href="/register?role=brand">
          Brandet képviselek →
        </Link>
      </Button>
    </div>

    {/* Stats */}
    <div className="mt-20 grid grid-cols-3 gap-8 text-white/80">
      <div>
        <div className="text-4xl font-bold text-accent">150+</div>
        <div className="text-sm">Aktív creator</div>
      </div>
      <div>
        <div className="text-4xl font-bold text-accent">30+</div>
        <div className="text-sm">Brand</div>
      </div>
      <div>
        <div className="text-4xl font-bold text-accent">100%</div>
        <div className="text-sm">Magyar</div>
      </div>
    </div>
  </div>
</section>
```


9.5 OG image és favicon

A generált `og-image.webp` kerüljön a `public/og-image.png`-be (konvertálva PNG-re).

Favicon:

- `public/favicon.ico` — generált logó kicsinyített verziója
- Vagy Claude Code készítsen SVG-t: egy stilizált "C" betű neon lime színnel sötét körben

9.6 Navbar és Footer komponens

`components/layout/Navbar.tsx` :

- Logo bal oldalt (szöveg: "creatorz" + lime accent dot)
- Középen menü: Creatorok / Hirdetések / Hogyan működik / Árak
- Jobb oldalt: bejelentkezve = dashboard link + avatar, kijelentkezve = Bejelentkezés + Regisztráció
- Mobil hamburger

`components/layout/Footer.tsx` :

- 4 oszlop: Termék / Cégeknek / Creatoroknak / Jogi
- Alul: copyright + social ikonok



9. FÁZIS ELLENŐRZÉSI PONT

- [] `npx tsx scripts/generate-images.ts` minden képet legenerál
- [] `public/images/generated/` mappa minden képpel feltöltött
- [] Landing page betöltődik, hero kép megjelenik
- [] OG image kép működik (Open Graph Debugger-rel teszteljük)
- [] Favicon megjelenik a böngészőfülön
- [] Navbar és Footer renderelődik
- [] Mobil layout működik

Ha mind ✓: "Mehet a 10. fázis"



10. FÁZIS — SOCIAL SCRAPE + CRON



Aktor: Claude Code



Időbecslés: 2-3 óra



Cél: Heti automatikus follower scrape Instagram, TikTok, Facebook (Pages) profilokon

10.1 Scraper functions

lib/scrapers/instagram.ts:

```
export async function scrapeInstagramFollowers(profileUrl: string): Promise<number | null> {
  try {
    const username = profileUrl.split("instagram.com/")[1]?.split("/")[0]?.split("?")[0];
    if (!username) return null;

    const response = await fetch(`https://www.instagram.com/${username}/`, {
      headers: {
        "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36",
      },
    });

    if (!response.ok) return null;
    const html = await response.text();

    // Match: "edge_followed_by":{"count":12345}
    const match = html.match(/"edge_followed_by":\{"count":(\d+)\}/);
    return match ? parseInt(match[1], 10) : null;
  } catch (err) {
    console.error("Instagram scrape failed:", err);
    return null;
  }
}
```

lib/scrapers/tiktok.ts:

```
export async function scrapeTikTokFollowers(profileUrl: string): Promise<number | null> {
  try {
    const response = await fetch(profileUrl, {
      headers: {
        "User-Agent": "Mozilla/5.0 (iPhone; CPU iPhone OS 14_0 like Mac OS X) AppleWebKit/
605.1.15",
      },
    });

    if (!response.ok) return null;
    const html = await response.text();

    // Match: "followerCount":12345
    const match = html.match(/"followerCount":(\d+)/);
    return match ? parseInt(match[1], 10) : null;
  } catch (err) {
    console.error("TikTok scrape failed:", err);
    return null;
  }
}
```

`lib/scrapers/youtube.ts` (a YouTube Data API ingyenes 10k quota/nap):

```
export async function fetchYouTubeSubscribers(channelUrl: string): Promise<number | null> {
  try {
    // Extract channel ID or username
    // ...
    // Use YouTube Data API v3 statistics endpoint
    const apiKey = process.env.YOUTUBE_API_KEY;
    if (!apiKey) return null;
    // ...
    return null; // Placeholder
  } catch {
    return null;
  }
}
```

10.2 Cron job

`app/api/cron/scrape-followers/route.ts`:

```
import { db } from "@lib/db";
import { creatorProfiles } from "@lib/db/schema";
import { eq, isNotNull } from "drizzle-orm";
import { scrapeInstagramFollowers } from "@lib/scrapers/instagram";
import { scrapeTikTokFollowers } from "@lib/scrapers/tiktok";

export async function GET(req: Request) {
  const authHeader = req.headers.get("authorization");
  if (authHeader !== `Bearer ${process.env.CRON_SECRET}`) {
    return new Response("Unauthorized", { status: 401 });
  }

  // Fetch all creators with social URLs
  const creators = await db.query.creatorProfiles.findMany({
    where: isNotNull(creatorProfiles.instagramUrl),
  });

  let updated = 0;
  for (const creator of creators) {
    const updates: Partial<typeof creatorProfiles.$inferInsert> = {};

    if (creator.instagramUrl) {
      const igCount = await scrapeInstagramFollowers(creator.instagramUrl);
      if (igCount !== null) {
        updates.instagramFollowers = igCount;
        updates.instagramLastChecked = new Date();
      }
    }

    if (creator.tiktokUrl) {
      const ttCount = await scrapeTikTokFollowers(creator.tiktokUrl);
      if (ttCount !== null) {
        updates.tiktokFollowers = ttCount;
        updates.tiktokLastChecked = new Date();
      }
    }

    if (Object.keys(updates).length > 0) {
      await db.update(creatorProfiles)
        .set(updates)
        .where(eq(creatorProfiles.id, creator.id));
      updated++;
    }

    // 2 sec delay to avoid rate limit
    await new Promise((r) => setTimeout(r, 2000));
  }

  return Response.json({ scraped: creators.length, updated });
}
```

10.3 Cron job manuális teszt

```
curl -H "Authorization: Bearer YOUR_CRON_SECRET" http://localhost:3000/api/cron/scrape-followers
```

Vagy a browseren keresztül egy védett admin oldalon "Manuális scrape futtatása" gomb.

10.4 Vercel Cron beállítása

A `vercel.json` már tartalmazza a cron-okat (6. fázisban beállítva).

👉 10. FÁZIS ELLENŐRZÉSI PONT

- [] Instagram scrape egy publikus tesztprofilon működik
- [] TikTok scrape egy publikus tesztprofilon működik
- [] Cron endpoint manuális tesztelése lefut és frissíti a DB-t
- [] Hibakezelés: ha egy scrape failel, a többi tovább megy
- [] `instagramLastChecked` timestamp frissül

Ha mind ✓: "Mehet a 11. fázis"

11. FÁZIS — POLISH, GDPR, LAUNCH

 **Aktor:** Claude Code

 **Időbecslés:** 4-6 óra

 **Cél:** Production-ready: GDPR megfelelés, ÁSZF, cookie banner, error handling, SEO

11.1 GDPR Cookie banner

`components/shared/CookieBanner.tsx` :

```

"use client";
import { useEffect, useState } from "react";
import { Button } from "@components/ui/button";
import Link from "next/link";

export function CookieBanner() {
  const [show, setShow] = useState(false);

  useEffect(() => {
    const consent = document.cookie.includes("creatorz_cookie_consent");
    if (!consent) setShow(true);
  }, []);

  function accept() {
    document.cookie = `creatorz_cookie_consent=accepted; max-age=31536000; path=/; samesite=lax`;
    setShow(false);
  }

  function reject() {
    document.cookie = `creatorz_cookie_consent=rejected; max-age=31536000; path=/; samesite=lax`;
    setShow(false);
  }

  if (!show) return null;

  return (
    <div className="fixed inset-x-0 bottom-0 z-50 border-t bg-background p-4 shadow-2xl">
      <div className="mx-auto flex max-w-7xl flex-col gap-4 md:flex-row md:items-center md:justify-between">
        <p className="text-sm text-muted-foreground">
          Ezen az oldalon sütiket használunk az élmény javításához és statisztikai célokra. A használatához beleegyezésed szükséges.{" "}
          <Link href="/cookies" className="underline">Részletek</Link>
        </p>
        <div className="flex gap-2">
          <Button variant="outline" size="sm" onClick={reject}>
            Elutasítás
          </Button>
          <Button size="sm" onClick={accept} className="bg-accent text-accent-foreground">
            Elfogadom
          </Button>
        </div>
      </div>
    </div>
  );
}

```

11.2 Jogi oldalak

Hozz létre 4 jogi oldalt:

app/(public)/aszf/page.tsx — Általános Szerződési Feltételek

Tartalma (sablon szöveg, később ügyvéddel pontosítsd!):

- Általános rendelkezések
- Szolgáltatás leírása
- Felhasználói típusok (creator és brand)
- Regisztráció
- Subscriptions és fizetési feltételek

- • Felhasználói tartalom és jogok
- • Adatvédelem (link a Privacy Policy-ra)
- • Felelősség kizárása
- • Megszűnés
- • Vitarendezés
- • Záró rendelkezések

`app/(public)/adatvedelem/page.tsx` — **Adatvédelmi tájékoztató**

- Adatkezelő (te / cég neve, székhely)
- Kezelt adatok típusa
- Adatkezelés jogalapja
- Adatok továbbítása (Stripe, Resend, Supabase, Vercel)
- Tárolási idő
- Felhasználói jogok (törlés, módosítás, hordozhatóság)
- Cookie használat
- Kapcsolatfelvétel (info@creatorz.hu)

`app/(public)/cookies/page.tsx` — **Cookie szabályzat**

`app/(public)/kapcsolat/page.tsx` — **Kapcsolat**

11.3 Account delete (GDPR Right to be forgotten)

`app/(dashboard)/[role]/settings/delete-account/page.tsx` :

- Megerősítő dialog
- Beleírja a usernevet, hogy biztosan ezt akarja
- Soft delete: user `suspended=true`, profil `hidden`
- 30 nap után hard delete (cron job)

11.4 Error handling

`app/error.tsx` :

```
"use client";
import { Button } from "@components/ui/button";

export default function Error({ error, reset }: { error: Error; reset: () => void }) {
  return (
    <div className="flex min-h-screen flex-col items-center justify-center gap-4 p-6 text-center">
      <h2 className="text-2xl font-bold">Valami hiba történt</h2>
      <p className="text-muted-foreground">Sajnáljuk a kellemetlenséget. Próbáld újra.</p>
      <Button onClick={reset}>Újra próbálkozás</Button>
    </div>
  );
}
```

`app/not-found.tsx` :


```
import Link from "next/link";
import { Button } from "@components/ui/button";

export default function NotFound() {
  return (
    <div className="flex min-h-screen flex-col items-center justify-center gap-4 p-6 text-center">
      <h1 className="text-6xl font-bold text-accent">404</h1>
      <h2 className="text-2xl font-bold">Az oldal nem található</h2>
      <p className="text-muted-foreground">
        Lehet, hogy a link elavult, vagy az oldal eltávolításra került.
      </p>
      <Button asChild>
        <Link href="/">Vissza a főoldalra</Link>
      </Button>
    </div>
  );
}
```

11.5 Loading states (skeleton)

app/(public)/creators/loading.tsx :

```
import { Skeleton } from "@components/ui/skeleton";

export default function Loading() {
  return (
    <div className="mx-auto max-w-7xl p-6">
      <Skeleton className="mb-6 h-12 w-64" />
      <div className="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
        {Array.from({ length: 9 }).map((_, i) => (
          <div key={i} className="rounded-xl border p-4">
            <Skeleton className="mb-4 h-16 w-16 rounded-full" />
            <Skeleton className="mb-2 h-6 w-32" />
            <Skeleton className="mb-4 h-4 w-24" />
            <Skeleton className="h-4 w-full" />
          </div>
        ))}
      </div>
    </div>
  );
}
```

11.6 SEO

app/sitemap.ts :

```
import { db } from "@lib/db";
import { creatorProfiles, users } from "@lib/db/schema";
import { eq, and } from "drizzle-orm";
import type { MetadataRoute } from "next";

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const baseUrl = process.env.NEXT_PUBLIC_APP_URL!;

  // Static pages
  const staticPages: MetadataRoute.Sitemap = [
    { url: baseUrl, lastModified: new Date(), priority: 1.0 },
    { url: `${baseUrl}/creators`, lastModified: new Date(), priority: 0.9 },
    { url: `${baseUrl}/ads`, lastModified: new Date(), priority: 0.9 },
    { url: `${baseUrl}/about`, lastModified: new Date(), priority: 0.5 },
    { url: `${baseUrl}/aszf`, lastModified: new Date(), priority: 0.3 },
    { url: `${baseUrl}/adatvedelem`, lastModified: new Date(), priority: 0.3 },
  ];

  // Creator profiles
  const creators = await db.query.creatorProfiles.findMany({
    columns: { username: true, updatedAt: true },
  });

  const creatorPages = creators.map((c) => ({
    url: `${baseUrl}/creators/${c.username}`,
    lastModified: c.updatedAt,
    priority: 0.7,
  }));

  return [...staticPages, ...creatorPages];
}
```

app/robots.ts :

```
import type { MetadataRoute } from "next";

export default function robots(): MetadataRoute.Robots {
  return {
    rules: [
      {
        userAgent: "*",
        allow: "/",
        disallow: ["/api/", "/admin/", "/creator/", "/brand/", "/login", "/register"],
      },
    ],
    sitemap: `${process.env.NEXT_PUBLIC_APP_URL}/sitemap.xml`,
  };
}
```

11.7 Performance optimalizáció

- Image optimization: minden `<Image>` `priority` csak fontosaknál
- Server Components default mindenhol
- "use client" csak ahol tényleg kell
- Database queries: `select` csak a szükséges oszlopokra
- Caching: `revalidate` beállítás public oldalakon (creator browse: 60 sec)

11.8 Domain verifikáció Resend-en

A Resend Dashboard → Domains → Add Domain → `creatorz.hu`

Add a Rackhost DNS-hez:

- TXT rekord: `@`, érték: `v=spf1 include:_spf.resend.com ~all`
- TXT rekord: `resend._domainkey`, érték: (Resend ad)
- MX rekord (csak ha Resend-en küldesz): `feedback-smtp.resend.com`, priority: 10

Várj kb. 15 percet, aztán Resend automatikusan verifikálja.

`EMAIL_FROM` változó: `Creatorz <hello@creatorz.hu>` (ahelyett, hogy `onresend.dev-domain` lenne).

11. FÁZIS ELLENŐRZÉSI PONT

- [] Cookie banner első látogatáskor megjelenik
- [] ÁSZF, Adatvédelmi, Cookie szabályzat oldalak elérhetőek
- [] Account delete működik (soft delete)
- [] 404 és error oldalak szépen néznek ki
- [] Loading skeletonok minden listanézeten
- [] Sitemap.xml elérhető (/sitemap.xml)
- [] Robots.txt elérhető (/robots.txt)
- [] Lighthouse score (Chrome DevTools): >85 Performance, >95 Accessibility, >90 SEO

Ha mind ✓: "Mehet a 12. fázis"



12. FÁZIS — DEPLOYMENT VERCEL-RE + DNS



Aktor: Claude Code + TE



Időbecslés: 1-2 óra



Cél: A creatorz.hu élesben fut a Vercel-en

12.1 Production environment változók

A Vercel Dashboard-on hozz létre egy új projektet:

- <https://vercel.com/new>
- Import Git Repository → válaszd a `creatorz` repot
- Framework: Next.js (auto-detect)
- Root directory: `./`
- **NE deploy-old még! Először állítsd be az env változókat:**

Environment Variables (Production):

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=...
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
SUPABASE_SERVICE_ROLE_KEY=...
DATABASE_URL=...

# Stripe (LIVE mode kulcsok!)
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_...
STRIPE_SECRET_KEY=sk_live_...
STRIPE_WEBHOOK_SECRET=... (új live webhook secret a production webhookhoz)
STRIPE_PRICE_CREATOR_MONTHLY=price_... (LIVE termékek price ID-ja!)
STRIPE_PRICE_FEATURE_7DAY=price_...
STRIPE_PRICE_FEATURE_30DAY=price_...

# Resend
RESEND_API_KEY=re_...
EMAIL_FROM=Creatorz <hello@creatorz.hu>

# Replicate
REPLICATE_API_TOKEN=r8_...

# App
NEXT_PUBLIC_APP_URL=https://creatorz.hu
NEXT_PUBLIC_APP_NAME=Creatorz

# Admin
ADMIN_EMAIL=a-te-email-cimed@gmail.com

# Cron
CRON_SECRET=... (generálj egy 64 karakteres random string-et)
```

FONTOS: Stripe LIVE kulcsokat csak akkor használj, amikor minden teszt sikeres! Indulásnál Test mode kulcsokkal indulj.

12.2 Stripe Live mode termékek

A Stripe Dashboard-on **Toggle Test/Live** → Live mode:

- Hozd létre újra a 3 terméket (Creator Monthly, 7day, 30day Feature) — most már LIVE-ban
- Másold ki az új Price ID-kat
- Vercel env változókba másold be

12.3 Webhook beállítása Live mode-ban

Stripe Dashboard → Live → Developers → Webhooks → Add endpoint:

- URL: `https://creatorz.hu/api/webhooks/stripe`
- Events: ugyanazok mint Test mode-ban (6.2)
- Mentsd el a Signing Secret-et → Vercel env: `STRIPE_WEBHOOK_SECRET`

12.4 Deploy

- Vercel Dashboard → projektnél: "Deploy" gomb
- Build várj kb. 3-5 percet
- Ha sikeres: megnyílik egy `creatorz-xyz.vercel.app` URL

Ellenőrizd:

- Landing page betöltődik
- Regisztráció működik
- Database migrations lefutottak (manuálisan Supabase-en)

12.5 Custom domain (creatorz.hu) hozzáadása

A Vercel projekt → Settings → Domains:

- Add: `creatorz.hu`
- Vercel megmondja a DNS rekordokat, amik kellenek:

```
A rekord:  
Name: @  
Value: 76.76.21.21  
  
CNAME rekord (www):  
Name: www  
Value: cname.vercel-dns.com
```

12.6 Rackhost DNS beállítása

- Lépj be: `https://my.rackhost.hu`
- Domain szolgáltatások → `creatorz.hu` → DNS szerkesztő
- **Adj hozzá A rekordot:**
 - Név: `@` (vagy üres)
 - Típus: A
 - IP cím: `76.76.21.21`
 - TTL: 3600
- **Adj hozzá CNAME rekordot:**
 - Név: `www`

- Típus: CNAME
- Érték: `cname.vercel-dns.com`
- TTL: 3600

- Mentés

Várj 10-30 percet, amíg a DNS propagálódik.

12.7 SSL aktiválás

A Vercel automatikusan kiállít egy Let's Encrypt SSL tanúsítványt — semmit nem kell csinálnod. Várj ~5 percet.

Ellenőrzés:

- <https://creatorz.hu> — működik HTTPS-sel
- A zöld lakat megjelenik a böngészőben

12.8 Final checklist

- [] <https://creatorz.hu> — betölt, HTTPS lakat zöld
- [] Új user regisztráció működik production-ben
- [] Email értesítés megérkezik
- [] Stripe Live mode-ban tesztel egy 100 Ft-os subscription-t (saját kártyával)
- [] Webhook fogadja az eseményt (Stripe Dashboard → Webhooks → log)
- [] Admin login működik (te magad)
- [] Cron job-ok megjelennek a Vercel-en (Settings → Cron Jobs)
- [] Sitemap.xml elérhető: <https://creatorz.hu/sitemap.xml>

12. FÁZIS ELLENŐRZÉSI PONT — LAUNCH!

Soft launch checklist (te végzed):

- [] 20-30 magyar UGC creator-t hívj meg személyesen (Facebook csoport, Discord, ismerősök)
- [] 5-10 márkát hívj meg személyesen
- [] Készíts 1-2 demo collaboration-t (te magad, hogy legyen review és featured creator)
- [] Posztold a saját social-jeidre
- [] Várj 1 hetet, gyűjts feedback-et

Ha minden ✓ → ÉLES VAGY! 



ZÁRÓ MEGJEGYZÉSEK A CLAUDE CODE-NAK

Általános elvek minden fázisra

- **Sose változtass tech stacket** kérdés nélkül
- **Minden Server Action zod-validált** legyen
- **Minden user-facing szöveg magyarul** legyen
- **Hibakezelés:** minden async művelet try/catch-ben, user-friendly hibaüzenettel
- **Mobile-first:** tervezz mobilra elsőként, asztali csak utána
- **Accessibility:** minden interaktív elem keyboard-elérhető, ARIA címkékkel
- **Type-safety:** NEM használj `any`-t — `unknown` + type narrowing
- **Performance:** Server Components default, "use client" csak ha kell
- **Sose commitolj** `.env.local` vagy bármely titkos kulcsot
- **Git commitek:** minden fázis végén szemantikus commit message

Ha hibába ütközöl

- NE találgass — ha valami nem világos, kérdezz a felhasználótól
- Logokat olvasd el (Vercel, Supabase, Stripe dashboard)
- TypeScript hibákat azonnal javítsd, ne hagyd
- Ha 3-szor sem sikerült megoldani, STOP és kérj segítséget

Tesztelés minden fázisban

A Claude Code minden fázis végén:

1. Futtassa: `npx tsc --noEmit` — TypeScript ellenőrzés
 2. Futtassa: `npm run lint` — ESLint ellenőrzés
 3. Futtassa: `npm run build` — production build próba
 4. Manuálisan tesztelje a kulcsfunkciókat
 5. Csak ezután mondja, hogy "Kész az X. fázis, várok ellenőrzésre"
-



Ha mind a 12 fázis ✓, akkor **a creatorz.hu élesben fut, kész a soft launchre.**

A következő hetek feladatai:

- Heti review feedback gyűjtés
- Bug fixek
- V2 feature-ök tervezése (OAuth verifikáció, in-app chat, mobile app)
- Marketing növelése

Sok sikert! Hajrá! 🚀

Dokumentum vége. Verzió 1.0 · 2026. május · Creatorz.hu Master Prompt File